Operations Management System

A Capstone Project Report of MSIT 5910

Submitted by:

Bilal Baheer, S552525

For the partial fulfilment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Stella Whyte

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

Term 1, October 2026

Abstract

Engineering firms that inspect piers, wharves, and bridges collect a lot of field data, but at mid-size firms that data ends up split across spreadsheets, shared drives, and document systems, so nobody can quickly see which structures need repair first or prove a record was reviewed. This project designed, built, tested, and planned the deployment of the inspection record management module of a cloud-based Operations Management System. The literature shows that a central relational database with consistently coded condition data is what keeps infrastructure information useful, but most research targets large agencies rather than mid-size consultancies. The module uses a web UI, an API gateway, an authentication service, and an inspection service with PostgreSQL storage and an event outbox. Its core algorithm validates each record, ranks it by its worst condition into IMMEDIATE, HIGH, ROUTINE, or MONITOR, and enforces a draft, submitted, approved review workflow where only engineers can approve. Testing at four levels passed all 46 automated tests and 11 of 12 acceptance tests, found and fixed five bugs, and documented four limitations, including a 100-record list cap. Under load the system read a record in 13 ms and saved one in 30 ms at the median, with zero errors in over 100,000 requests, about 120 MB of memory per service, and 100% data consistency. A heuristic usability review raised the interface score from 3.2 to 4.0 out of 5. The report ends with a Docker and Azure Container Apps deployment plan and a maintenance plan.

Acknowledgements

I would like to thank Dr. Stella Whyte for her guidance and detailed feedback throughout this capstone, especially on referencing, figure numbering, and evidence, which made this report better. I also thank the Department of CS & MSIT and the University of the People for a program that let me build something real while I learned.

Table of Contents

Abstract 2
Acknowledgements 3
Table of Contents 4
List of Tables 6
List of Figures 7
Abbreviations and Symbols 8
Chapter 1: Introduction 9
1.1 Background 9
1.2 Problem Statement 9
1.3 Stakeholders 10
1.4 Objectives 10
1.5 Research Questions 11
1.6 Scope 11
1.7 Organization of the Report 11
Chapter 2: Literature Review 12
2.1 Database-Centered Infrastructure Management 12
2.2 Managing Defect and Condition Data 12
2.3 Fragmentation as the Main Barrier 13
2.4 Requirements, Architecture, and Design Practice 13
2.5 Testing and Quality 14
2.6 DevOps, Deployment, and Maintenance 15
2.7 Security and Ethics 15
2.8 Summary and Research Gap 16
Chapter 3: System Design and Methodology 17
3.1 Development Approach 17
3.2 Requirements 17
3.3 System Architecture 18
3.4 Algorithms 20
3.5 Data Design 21
3.6 Tools and Technologies 21
3.7 Testing Methodology 22
3.8 Security and Ethics by Design 22
3.9 Project Timeline 23

3.10 Design Evolution From the Unit 3 Plan 24
Chapter 4: Results and Evaluation 25
4.1 Core Algorithm Implementation 26
4.2 System Integration Summary 28
4.3 Output and Screenshots 31
4.4 Testing Results 35
4.5 Performance Evaluation 45
4.6 Usability Evaluation 48
4.7 Summary of Bugs and Limitations 51
Chapter 5: Discussion 52
5.1 Interpretation of Results 52
5.2 Comparison With the Literature 53
5.3 Deployment and Configuration 54
5.4 Maintenance Plan 57
5.5 Limitations 58
5.6 Evaluation Against the Objectives 59
Chapter 6: Conclusion and Future Work 59
6.1 Summary of Main Findings 59
6.2 Answers to the Research Questions 60
6.3 Contributions 60
6.4 Lessons Learned and Challenges 60
6.5 Future Work 61
References 62

List of Tables

Table 1.1. Project Objectives

Table 2.1. Summary of Key Sources and Their Influence on the Project

Table 3.1. Functional Requirements

Table 3.2. Non-Functional Requirements

Table 3.3. Modules of the Integrated System

Table 3.4. Repair Priority Decision Rules (Checked Top to Bottom)

Table 3.5. Tools and Technologies

Table 3.6. Testing Levels

Table 3.7. Project Phases and Status

Table 3.8. Planned Design Versus What Was Built

Table 4.1. Tagged Milestones

Table 4.2. Automated Test Results (v1.1.1)

Table 4.3. Requirement Traceability and Results

Table 4.4. Test Summary Report (Unit 7)

Table 4.5. Benchmarking Conditions (Unit 6 Load Test)

Table 4.6. Load Test Results (10 Connections, 15 Seconds per Scenario)

Table 4.7. Data Consistency Checks

Table 4.8. Heuristic Usability Review

Table 4.9. Bugs, Issues, and Limitations Found by Testing

Table 5.1. Deployment Options Compared

Table 5.2. Environment Variables

Table 5.4. Post-Deployment Risks and Mitigation

Table 5.3. Objectives Versus Results

List of Figures

Figure 3.1. Integrated Architecture of the OMS Inspection Module

Figure 3.2. Branches, Tags, and Commit History

Figure 4.1. The validateInspection() Function

Figure 4.2. Repair Priority and Workflow Rules

Figure 4.3. The createInspection() Service Method

Figure 4.4. Token and Role Check in the inspection-service

Figure 4.5. Transaction in the PostgreSQL Repository

Figure 4.6. The Outbox Publisher

Figure 4.7. Wiring the Modules Together in server.js

Figure 4.8. Demo Data Created Through the API

Figure 4.9. Login Screen

Figure 4.10. Inspector Dashboard After Saving an Inspection With a Critical Finding

Figure 4.11. Engineer Dashboard With Approve and Send Back Actions

Figure 4.12. Read-Only Manager View

Figure 4.13. End-to-End Input and Output Through the Gateway

Figure 4.14. Unit and API Tests After the Fix (36 Passed)

Figure 4.15. Integration Tests Against PostgreSQL and auth-service Tests

Figure 4.16. Combined Unit and Integration Coverage for the inspection-service (v1.1.0)

Figure 4.17. Failed Test Before the Fix: Expected 403, Received 409

Figure 4.18. Failed Test on the Pre-Fix Commit: Non-UUID ID Crashes the Query

Figure 4.19. Acceptance Test Results After the Fix

Figure 4.20. 60-Second Stability Test

Figure 4.21. Data Consistency After the Stability Test

Figure 4.22. Volume Check: List Capped at 100 Records

Figure 4.23. Load Test Summary From perf/results/results.json

Figure 4.24. Response Latency by Scenario (ms)

Figure 4.25. Throughput by Scenario (Requests per Second)

Figure 4.26. Memory Footprint per Service, Idle and Peak Under Load (MB)

Figure 4.27. Validation Messages Before the Fix

Figure 4.28. Validation Messages After the Fix

Abbreviations and Symbols

API: Application Programming Interface

AT: Acceptance Test

CI: Continuous Integration

FR / NFR: Functional / Non-Functional Requirement

JWT: JSON Web Token

ms: Milliseconds

OMS: Operations Management System

p50, p90, p99 — 50th (median), 90th, and 99th percentile latency

RBAC: Role-Based Access Control

req/s: Requests per second

SQL: Structured Query Language

UI: User Interface

UUID: Universally Unique Identifier

WSL: Windows Subsystem for Linux

Introduction

1.1 Background

Public infrastructure in the United States is old and getting older, and the job of figuring out how bad things are falls to engineering firms that send inspectors into the field. Inspectors climb piers, walk bridge decks, and crawl under wharves with cameras, tablets, and sometimes paper forms. What they collect is valuable data about structures that people drive over and ships tie up against. This capstone is about what happens to that data after the inspector drives home.

At a lot of mid-size firms, and I have watched this happen at work, the information splits apart right away. Photos go into a shared drive organized by project number. Condition ratings get typed into a spreadsheet one person maintains. The signed report goes into a document system that can be searched by client but not by which pier it covers. Each tool is fine on its own, but together they mean nobody can quickly answer a simple question like “which structures on this project need repair first?” or “how has this wharf changed over six years?” Poor information handling is a real cost driver on projects, since future estimates depend on knowing what happened on past jobs (Oguz, 2025).

I like building software and tools to solve a problem and decided to build a cloud-based Operations Management System. For this capstone I designed, built, tested, and planned the deployment of one module of that system: inspection record management.

1.2 Problem Statement

Inspection and rehabilitation data at mid-size engineering firms is spread across disconnected spreadsheets, shared drives, and document systems. This causes duplicate data entry, slow reporting, no reliable way to see which structures need repair first, and no guarantee

that a record was reviewed before someone acted on it. This project builds a working, cloud-ready module that stores inspection records in one place, ranks them by repair urgency, and enforces a review workflow, so a project manager can find what matters in seconds instead of digging through several systems.

1.3 Stakeholders

Field inspectors are the ones typing the same data more than once, so one system directly cuts their paperwork. Engineers review and approve records and need the worst problems at the top of their list. Project managers need accurate status to keep jobs on schedule. Asset owners, such as port authorities and city departments, make repair decisions from the final reports. The IT and security team inherits responsibility for anything that stores client data. The public is an indirect stakeholder, since better-tracked inspection data means unsafe structures get flagged sooner.

1.4 Objectives

The objectives below come from my Unit 2 proposal, tightened so each one can be measured. Table 1.1 lists them, and Chapter 5 reports how far each was met.

Table 1.1

Project Objectives

Objective	**Measured**
Build a REST API that creates, reads, lists, filters, and moves inspections through review, with role-based access, passing automated tests	Endpoints delivered; automated test pass rate
Rank every inspection by repair urgency so the worst condition always comes first	Priority rules verified by unit and acceptance tests
Let a user find the records they need through the UI using at least three filters, in well under one minute	API latency; filters available; timed user test
Build security and ethics in from the start, with every stored data element mapped to a control	Security review; access-control acceptance tests

Objective	Measured
Keep the system fast, stable, and consistent under load on small hardware	Latency, throughput, error rate, memory, row counts
Make the system deployable and maintainable by someone other than me	Reproducible setup steps; deployment and maintenance plan

1.5 Research Questions

Three questions guided the work. First, what does existing research say about organizing inspection and condition data so it stays useful over an asset's life? Second, what would a minimal, practical module look like for a mid-size firm that can't justify a full enterprise asset management platform? Third, how can security, testing, and maintainability be built into the design from the start instead of patched on later?

1.6 Scope

In scope is one module of the larger OMS: a web UI, an API gateway, an authentication service, an inspection service with repair-priority and review-workflow rules, PostgreSQL storage, an event outbox for other OMS services, automated tests at several levels, Docker packaging, and a documented Azure deployment and maintenance plan. Out of scope are offline field capture, automated condition rating with machine learning, billing, document storage, 3D models, migration of old records, and multi-client data separation, which will come with the production rollout. Those exclusions are listed on purpose so the project could finish on time (Oguz, 2025).

1.7 Organization

Chapter 2 reviews the research and industry sources behind the design. Chapter 3 explains the requirements, architecture, algorithms, tools, and testing method. Chapter 4 presents what was built and every test result, including the Unit 7 system and acceptance testing. Chapter

5 interprets the results, compares them with the literature, covers deployment and maintenance, and states the limitations. Chapter 6 summarizes the contributions and future work.

Chapter 2: Literature Review

A literature review should do more than list sources. It should connect them, show what is settled, and point out what is not (Sheppard, 2019). This chapter moves from how infrastructure inspection data is managed in research, to the software engineering methods I used, and ends with the gap this project fills.

2.1 Database Centered Infrastructure

The clearest theme in recent work is that a well-structured relational database, not a fancy interface, is what makes infrastructure data useful. Pallante et al. (2024) built an integrated system for managing bridges across the Lazio region of Italy, and they treat the relational database as the hub for everything else. Inspection findings, design documents, and survey data all feed into one queryable database, and from there the same information drives maps and building information models. Their methodology depends on keeping that database at the center so information stays current across a bridge's life.

They also admit a limitation that matters for my project. The approach needs a minimum set of starting data, and in practice a lot of it isn't digital and has to be entered by hand. That is exactly the situation at a mid-size firm. The data exists, but it lives in PDFs and spreadsheets, and someone has to do the boring work of structuring it before anything advanced is possible.

2.2 Managing Defect and Condition Data

Li et al. (2023) came at the same problem from the defect side. Their bridge maintenance system classifies and codes defects from inspections, stores them in a database, grades the

bridge's technical condition, and suggests maintenance based on the score. Jeon et al. (2023) extended this to existing bridges, which is harder because the as-built information is often incomplete. Read together, these papers show that the value isn't the 3D model. It is condition data coded consistently enough to compare across inspections and assets, which then drives a decision about what to fix first. My repair-priority algorithm is a small, practical version of that idea.

2.3 Fragmentation as the Main Barrier

Through this research, fragmented information keeps showing up as the obstacle. Pallante et al. (2024) frame their whole contribution as a response to data scattered across formats and departments, and Li et al. (2023) start from the fact that defects pile up when findings aren't tracked over time. The pattern is consistent enough that fragmentation looks like the default state of inspection data unless someone designs against it.

2.4 Requirements, Architecture, and Design Practice

The software engineering readings supply the methods for designing against it. Letaw (2024) separates functional requirements, which say what a system does, from nonfunctional requirements, which say how well it does it, and points out that "each nonfunctional requirement has a quantity. That helps make it testable." Summers (2020) adds that "each of the system and subsystem requirements should be assigned a project-unique identifier to support testing and enhance traceability." I followed both rules, which later made it possible to trace every test back to a requirement.

For architecture, Walker (2022) is useful because he gives the downside of every pattern, not just the upside. He says the layered pattern makes it easy to write applications quickly, "but a disadvantage is that it can be hard to split up the layers later," and that with microservices "the

complexity means you need greater architectural expertise to make everything work correctly." Those trade-offs shaped my choice to keep the module to three small services. The Bit.ai Editorial Team (2025) describes a design document as "kind of like a blueprint for your software project," which is how I used Chapter 3 of this report.

2.5 Testing and Quality

BrowserStack (2025) describes system testing as testing "the entire software product as a whole to check whether everything works together correctly, just like a real user would use it," and says it is done when exit criteria are met, "not when you run out of time." Hrupa (2024) explains that system testing checks the whole integrated application against its requirements and comes before user acceptance testing. Letaw (2024) recommends writing acceptance criteria in a given-when-then format, which "is used for writing user story acceptance criteria." Wells (2025) and LaunchNotes (2023) both argue for combining quantitative and qualitative metrics, and Nielsen's (1994) usability heuristics give a structured way to judge an interface without a large user study.

The readings also explain why one level of testing is never enough. Amazon Web Services (n.d.) says a unit test "needs to run in isolation," which keeps it fast and stable but means it can't see how parts behave together. GeeksforGeeks (2025) states the flip side directly, warning that unit tests "cannot detect integration issues between different modules or services." Summers (2020) adds that "software engineering design, code and unit test, plans, and test procedures integrated with applied software tell us that the software is correctly developed," which I read as a reminder that confidence comes from all of these together. That is why I planned unit, integration, system, and acceptance levels instead of relying on one.

2.6 DevOps, Deployment, and Maintenance

Red Hat (2025) describes DevOps as speeding up how an idea gets from development to production, with the mantra “you build it, you run it,” and says containers “make it easier to move applications between development, testing, and production environments.” Sumo Logic (2025) separates release from deployment, noting that “a software release is a specific version of a code and its dependencies made available for deployment,” and that continuous deployment works best for teams that have “invested heavily in automated testing.” The twelve-factor approach says configuration belongs in environment variables, not code (Wiggins, n.d.). Davies (2024) covers the Git practices behind all of this, including branches as “independently developed sequences of changes for some specific purpose.” For maintenance after release, Thales (n.d.) defines four types (corrective, adaptive, perfective, and preventive) and warns that maintenance can cost “up to two-thirds of the entire software process cycle.”

2.7 Security and Ethics

Because the system holds client data about public structures, security is a core topic. The OWASP Foundation (2021) ranks broken access control as the most common risk category in web applications. It is usually the ordinary mistake of checking permissions in the interface and forgetting to check them on the server. The National Institute of Standards and Technology (NIST, 2020) recommends least privilege, and the Association for Computing Machinery (2018) treats avoiding harm and respecting privacy as basic professional duties. For passwords, the OWASP Foundation (n.d.) recommends slow, salted hashing such as bcrypt so stolen hashes are hard to crack.

2.8 Summary and Research Gap

Table 2.1 summarizes the main sources and what each one contributed to this project.

Table 2.1

Key Sources

Source	**Main idea**	**How it shaped this project**
Pallante et al. (2024)	A central relational database drives infrastructure management	One PostgreSQL store for all inspection records
Li et al. (2023); Jeon et al. (2023)	Coded defect data drives condition grades and maintenance decisions	Severity-coded findings feed the repair-priority algorithm
Letaw (2024); Summers (2020)	Testable, uniquely identified requirements	FR and NFR IDs traced to every test
Walker (2022)	Patterns have real trade-offs	Three small services, a gateway, and an event outbox
BrowserStack (2025);	System and acceptance testing against requirements	Four test levels with exit criteria
Red Hat (2025); Sumo Logic (2025); Wiggins (n.d.)	Containers, releases, and configuration as code	Docker images, Git-tag releases, environment variables
Thales (n.d.)	Four types of maintenance	Maintenance plan in Section 5.4
OWASP Foundation (2021, n.d.); NIST (2020);	Access control, hashing, least privilege, ethics	Server-side role checks, bcrypt, synthetic data only

Put together, a gap shows up. The infrastructure research focuses on large, agency-level systems like regional bridge portfolios and digital twins, which assume a client with a dedicated asset management budget. What's missing is the mid-size engineering consultancy that inspects structures for many clients, can't buy a big platform, and falls back to spreadsheets, which is exactly the fragmentation the research warns about. The DevOps and testing sources also tend to present gateways, password hashing, and containers as clear wins without showing what they cost on small hardware. This project fills both gaps with a small, tested, measured module that a team like mine can actually adopt.

Chapter 3: System Design and Methodology

3.1 Development Approach

I built the module in short, planned increments across the course units, with each unit ending in a tagged release. Unit 3 produced the requirements and design. Unit 4 implemented the core logic. Unit 5 added unit tests, a CI workflow, and Git Flow branching. Unit 6 integrated the full system and measured its performance. Unit 7 added system and acceptance testing and the maintenance plan. All work happened on feature branches merged into develop, then released to main with a tag (Figure 3.2).

3.2 Requirements

Requirements came from document analysis of existing inspection reports and conversations with colleagues who do inspection and review work. Each requirement has a unique ID so it can be traced to a test, as Summers (2020) recommends. Tables 3.1 and 3.2 show the final requirements for the version that was built.

Table 3.1

Functional Requirements

#	**Requirement**
FR-01	An inspector or engineer can create an inspection with project, structure, date, a condition rating from 1 to 5, and zero or more findings.
FR-02	The system rejects an invalid inspection and lists every problem at once.
FR-03	The system calculates a repair priority (IMMEDIATE, HIGH, ROUTINE, MONITOR) from the worst condition in the record.
FR-04	The list shows the most urgent records first and can be filtered by project, priority, and status.
FR-05	Records move from draft to submitted to approved, a submitted record can be sent back to draft, and approved records are locked.
FR-06	Inspectors create and submit, only engineers approve, and managers have read-only access, all checked on the server.

#	**Requirement**
FR-07	Every create and status change writes an event to the outbox for other OMS services.
FR-08	Users log in with email and password and get a one-hour token; a wrong email and a wrong password get the same error.
FR-09	An inspector can edit a draft before submitting it (planned for v1.2.0).

Table 3.2

Non-Functional Requirements

#	**Category**	**Requirement**
NFR-01	Performance	Reading or saving one record has a median latency under 100 ms at 10 concurrent connections
NFR-02	Reliability	0% error rate under load and no lost or partial records
NFR-03	Security	Every request is authorized on the server; tokens are signed, carry an issuer, and expire in one hour; no secrets in Git
NFR-04	Usability	Average score of at least 4.0 out of 5 on the heuristic review
NFR-05	Resources	Each service uses under 256 MB of memory under load
NFR-06	Maintainability	Storage can be swapped without changing business logic
NFR-07	Retrieval	A manager can find any inspection in well under one minute

3.3 System Architecture

Figure 3.1 shows the architecture. The browser only talks to the web gateway on port 3000. The gateway serves the UI and forwards /api/auth requests to the auth-service and /api/inspections requests to the inspection-service. Each service owns its own database, which follows the microservices idea that components are “independent from each other” (Red Hat, 2025). The inspection-service also writes events to an outbox table so a relay can forward them to Azure Service Bus later without losing anything, which Richardson (n.d.) calls the transactional outbox pattern. Table 3.3 sums up what each part does.

The most important decision is where the trust line sits. Everything in front of the services is untrusted, including my own UI. The UI hides buttons a user can’t use, but that is just

for a clean screen. The real permission check happens again on the server for every request, because broken access control is the top web risk (OWASP Foundation, 2021).

Figure 3.1

Architecture of the OMS

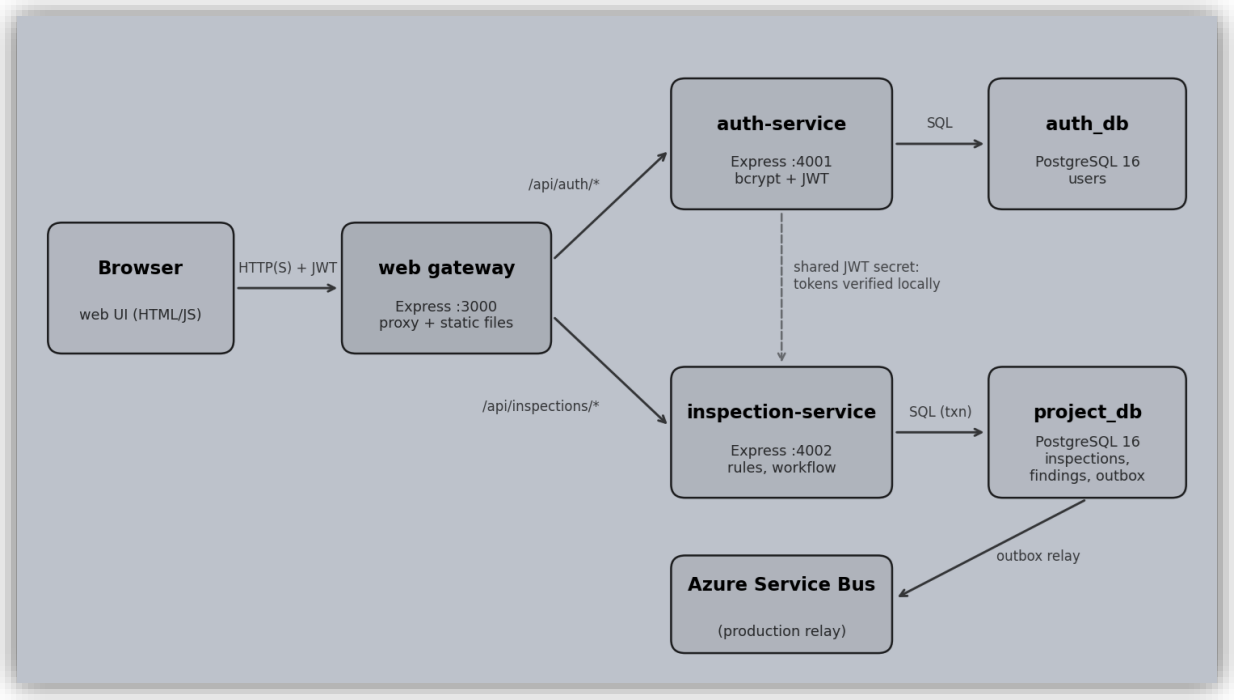


Table 3.3

Modules

Module	**Technology**	**Main responsibility**
Web UI	HTML, CSS, JavaScript	Login, inspection form, prioritized list, filters, submit and approve buttons
Web gateway	Express, http-proxy-middleware	Serves the UI and routes API calls to the right service
auth-service	Express, bcryptjs, jsonwebtoken	Password check and a one-hour JWT carrying the user role
inspection-service	Express, pg, jsonwebtoken	Validation, repair priority, review workflow, storage, outbox events
auth_db, project_db	PostgreSQL 16	Users; inspections, findings, and outbox events

3.4 Algorithms

The heart of the module is three pure functions in inspectionRules.js with no database or web code inside, so they are easy to test and reuse.

Validation. validateInspection() checks that the project and structure are present, the rating is a whole number from 1 (Critical) to 5 (Good), the date is real and not in the future, and every finding names an element and uses a known severity. It collects every problem and returns them all at once, so the user can fix everything in one try.

Repair priority. calculatePriority() checks the most urgent conditions first and stops at the first match (Table 3.4). Checking from worst to best means a structure with a good overall rating still gets flagged if one element is critical, which is how an engineer thinks.

Table 3.4

Repair Priority

Order	**Condition**	**Priority**
1	Rating is 1, or any finding is critical	IMMEDIATE
2	Rating is 2, or two or more findings are major	HIGH
3	Rating is 3, or exactly one finding is major	ROUTINE
4	Anything else	MONITOR

Review workflow canTransition() allows draft to submitted, submitted to approved, and submitted back to draft, and nothing out of approved. It returns false for an unknown status instead of crashing. The service layer adds the rule that only engineers can approve, and it always takes the inspector’s ID from the verified token, never from the request body, so nobody can file a record under someone else’s name.

3.5 Data Design

project_db has three tables. inspections holds one row per inspection with its rating, calculated priority, status, and inspector ID, and a CHECK constraint keeps the rating between 1 and 5 even if the API is bypassed findings holds one row per finding linked to its inspection. outbox_events stores each event's type and a JSON payload until a relay publishes it. An inspection and its findings are saved in one transaction so a crash can't leave half a record. auth_db has a users table with email, role, and a bcrypt hash. Only synthetic demo data is used, so no client or personal information is ever stored.

3.6 Tools and Technologies

Table 3.5 lists the main tools. I picked the same stack the real OMS team already uses, so the module can be handed over without a rewrite, and I kept the list short on purpose so one developer could understand every part.

Table 3.5

Tools and Technologies

Topic	**Tool**	**Reasons**
Runtime and API	Node.js 20, Express 5	Same stack as the real OMS team, so the module can be handed over
Database	PostgreSQL 16, pg	Relational data with joins, constraints, and transactions
Security	bcryptjs, jsonwebtoken	Slow salted password hashes; signed, expiring tokens
Gateway	http-proxy-middleware	One entry point; internal addresses stay hidden
Testing	Jest, Supertest, autocannon	Unit, API, integration, acceptance, load, and stability tests
Version control and CI	Git, GitHub, GitHub Actions	Git Flow branches, tags, tests on every push
Packaging and cloud	Docker, Docker Compose, Azure Container Apps (planned)	Same image locally and in the cloud

3.7 Testing Methodology

Testing was planned in four levels, each one catching what the level below can't (Table 3.6). Every acceptance test carries the ID of the requirement it checks, and exit criteria were set before testing started.

Table 3.6

Testing Levels

Level	What it checks	How
Unit and API	Rules, service layer, routes	Jest with fakes for the database; Supertest for HTTP
Integration	Real database behavior: transactions, constraints, outbox	Jest against a freshly migrated PostgreSQL 16 database
System	Performance, stability, data consistency, volume	autocannon load and 60-second soak tests; SQL row counts
Acceptance	Each requirement from a user's point of view	12 Given/When/Then tests through the gateway
Usability	Whether the UI makes sense to an inspector	

3.8 Security and Ethics by Design

In Unit 2, I reviewed the ethical and security risks before writing any code, and the design answers each one. The first ethical issue is privacy. Real inspection files can contain inspector names, GPS coordinates, and photos that catch people or private property in the background, so the prototype only ever uses synthetic data, which removes the privacy risk instead of just managing it. The second is consent and transparency: anyone who tries the system in a pilot will know their use is being observed and that feedback may appear in this report. The third is intellectual property, so every library is open source and listed in package-lock.json. The fourth is fairness. If priorities were set by a model trained on past inspections, a structure could be flagged or ignored partly because of who inspected it, so the priority rule is a simple, visible

table that anyone can check (Table 3.4). This fits the ACM's call to avoid harm and respect privacy (Association for Computing Machinery, 2018).

On the security side, every request is authorized on the server, following least privilege (NIST, 2020). Tokens are signed, carry an issuer, and expire after one hour. The inspector ID always comes from the token, never the request body. Wrong emails and wrong passwords get the same error so attackers can't discover accounts. Passwords are stored as bcrypt hashes (OWASP Foundation, n.d.), database queries are parameterized against injection, the UI escapes anything a user types to block cross-site scripting, and secrets live in environment variables and Key Vault, never in Git. Chapter 4 tests several of these directly, including forged and expired tokens in AT-10.

3.9 Project Timeline

The project followed the twelve-week plan from Unit 2, with milestones at requirements sign-off, design approval, feature freeze, and final submission. Table 3.7 summarizes how the phases mapped to course units. The only phase that slipped is cloud deployment, for the credit reasons explained in Chapter 5.

Table 3.7

Phases and Status

Phase	**Units**	**Output**
Problem definition and planning	1 to 2	Problem statement, SMART goals, Gantt chart, feasibility checklist
Requirements and design	3	Literature review, requirements, architecture, technical report
Core logic	4	Validation, priority, and workflow functions; CI/CD plan
Unit testing and version control	5	32 unit tests, three bugs fixed, Git Flow, tags
Integration and performance	6	Full system, PostgreSQL, gateway, auth, load tests, Docker

Phase	Units	Output
System testing and maintenance plan	7	Acceptance and stability tests, B-05 fix, maintenance plan
Cloud deployment and final report	8	Azure deployment session, timed user test, final report

3.10 Design Evolution from the Unit 3 Plan

The design I proposed in Unit 3 was broader than what I built, and I want the report to be honest about that. Some features were cut to fit one part-time developer and the course timeline, and others changed once I started building (Table 3.8). The core goals of one store, clear priority, and an enforced review did not change.

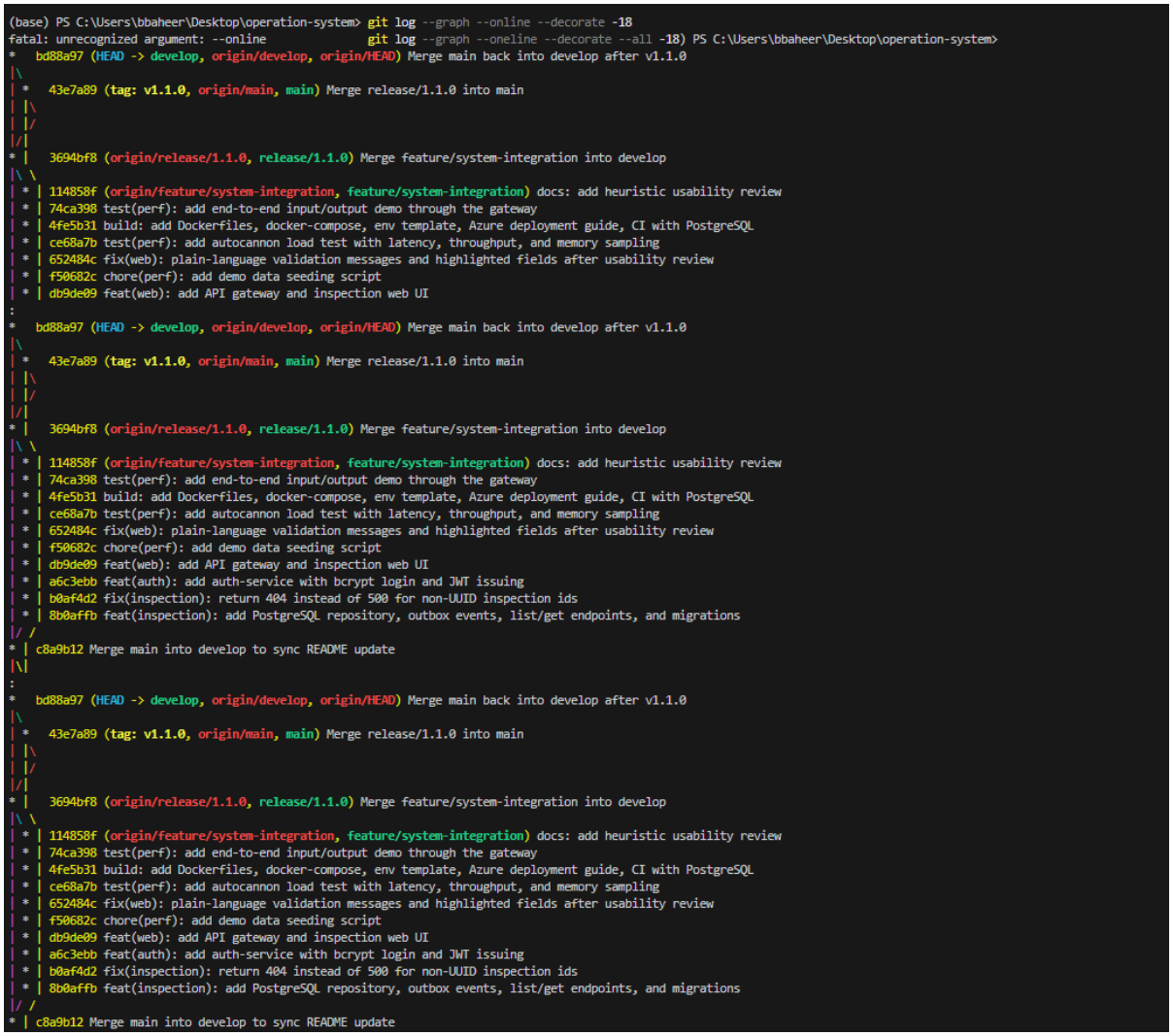
Table 3.8

Planned Design Versus What Was Built

Unit 3 plan	Built in v1.1.1	Reason
React single-page app	Plain HTML and JavaScript behind one api() helper	Smaller for a capstone; can be swapped for the team's React app later
Six services (auth, records, search, audit, export, 3D)	Gateway, auth-service, inspection-service (search inside it)	Scope for one developer; Walker's (2022) warning about microservice complexity
Four roles	Inspector, engineer, manager (read-only)	Matches who records, who approves, and who only reads
Append-only version history	Outbox event for every change; approved records locked	Covers the audit need for the workflow; full field history deferred
Firm SSO login	Own auth-service issuing JWTs	Keeps the prototype independent; SSO planned for production
Document links, multi-client scoping, export, 3D models	Not built	Deferred to later releases

Figure 3.2

Branches, Tags, and Commit History



Chapter 4: Results and Evaluation

This chapter shows what I built, what it looks like running, and every test result. The code is at https://github.com/BilalBaheer/operation-system. Results up to Unit 6 refer to tag v1.1.0, and the Unit 7 testing fix is v1.1.1.

4.1 Core Algorithm Implementation

Figure 4.1 shows validateInspection() collecting every problem before returning. Figure 4.2 shows calculatePriority() checking from most to least urgent, along with the workflow rules. Figure 4.3 shows the service method that validates, calculates the priority, saves the draft, and publishes an event, taking the inspector ID only from the token.

Figure 4.1

The validateInspection() Function

```
 Returns { valid: boolean, errors: string[] }
 */
function validateInspection(input) {
  const errors = [];

  if (!input.projectId) errors.push('projectId is required');
  if (!input.structureId) errors.push('structureId is required');

  const rating = input.conditionRating;
  if (!Number.isInteger(rating) || rating < 1 || rating > 5) {
    errors.push('conditionRating must be a whole number from 1 to 5');
  }

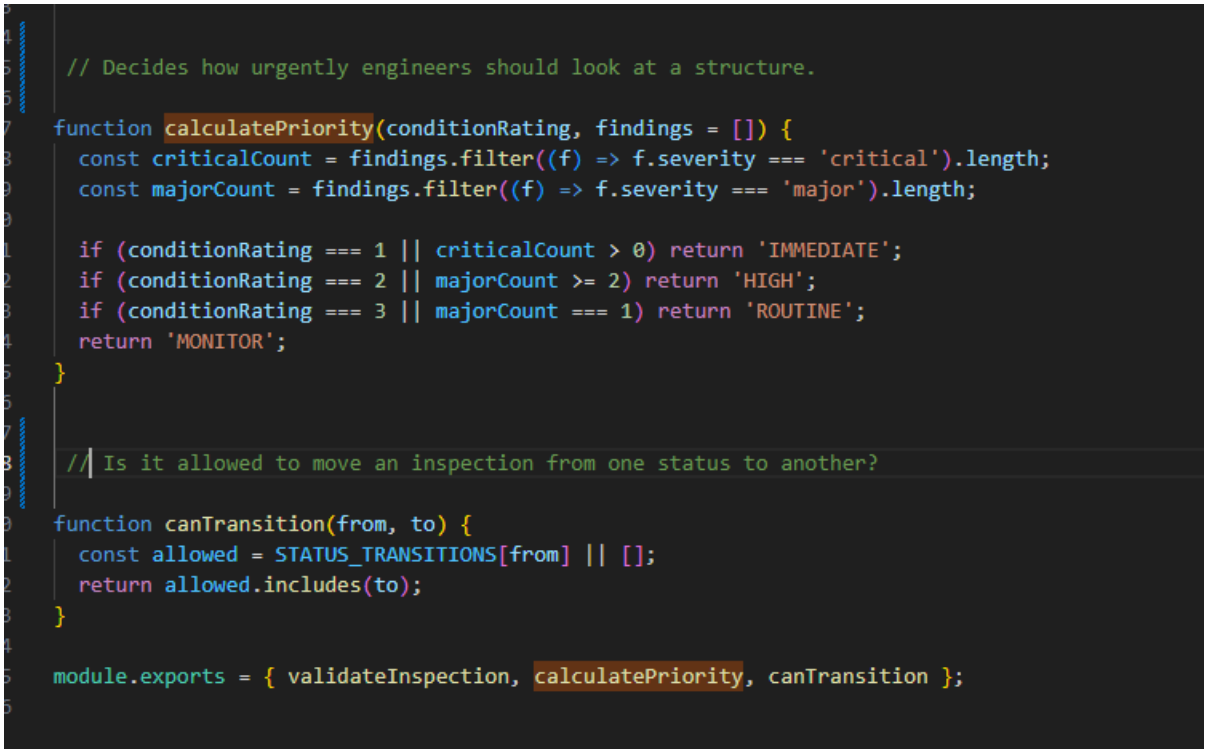
  if (!input.inspectionDate || Number.isNaN(Date.parse(input.inspectionDate))) {
    errors.push('inspectionDate must be a valid date');
  } else if (new Date(input.inspectionDate) > new Date()) {
    errors.push('inspectionDate cannot be in the future');
  }

  (input.findings || []).forEach((f, i) => {
    if (!f.element) errors.push(`findings[${i}].element is required`);
    if (!SEVERITIES.includes(f.severity)) {
      errors.push(`findings[${i}].severity must be one of ${SEVERITIES.join(', ')}`);
    }
  });

  return { valid: errors.length === 0, errors };
}
```

Figure 4.2

Repair Priority and Workflow Rules



```
// Decides how urgently engineers should look at a structure.

function calculatePriority(conditionRating, findings = []) {
  const criticalCount = findings.filter((f) => f.severity === 'critical').length;
  const majorCount = findings.filter((f) => f.severity === 'major').length;

  if (conditionRating === 1 || criticalCount > 0) return 'IMMEDIATE';
  if (conditionRating === 2 || majorCount >= 2) return 'HIGH';
  if (conditionRating === 3 || majorCount === 1) return 'ROUTINE';
  return 'MONITOR';
}

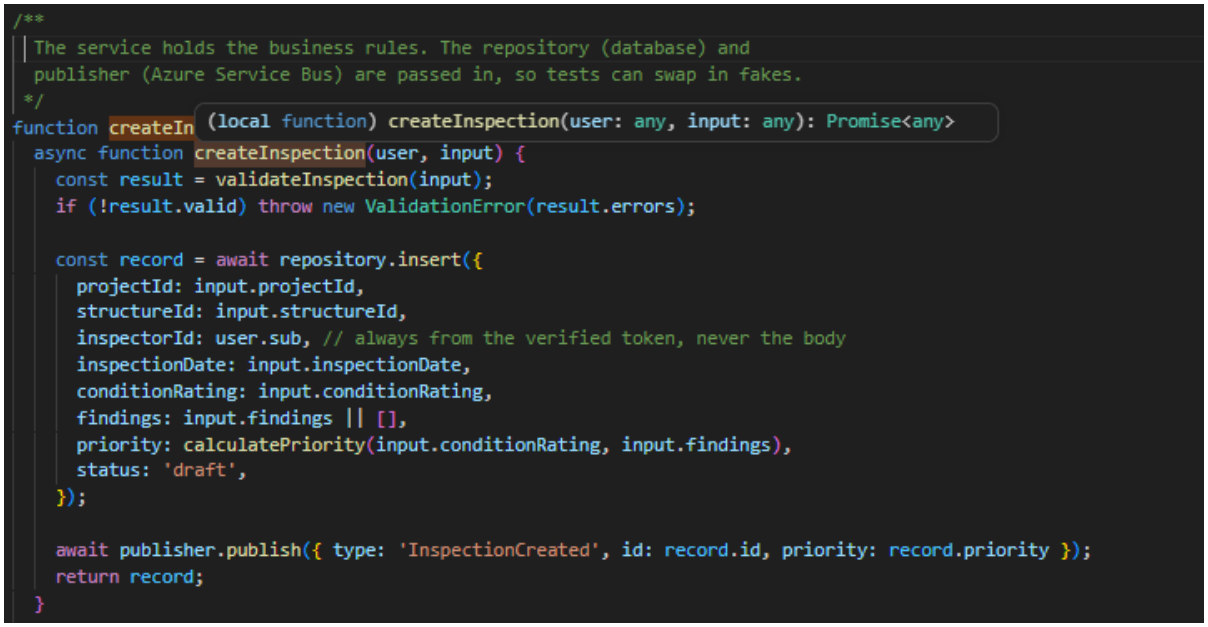

// Is it allowed to move an inspection from one status to another?

function canTransition(from, to) {
  const allowed = STATUS_TRANSITIONS[from] || [];
  return allowed.includes(to);
}

module.exports = { validateInspection, calculatePriority, canTransition };
```

Figure 4.3

The createInspection() Service Method



```
/**
 The service holds the business rules. The repository (database) and
 publisher (Azure Service Bus) are passed in, so tests can swap in fakes.
*/
function createIn  (local function) createInspection(user: any, input: any): Promise<any>
  async function createInspection(user, input) {
    const result = validateInspection(input);
    if (!result.valid) throw new ValidationError(result.errors);

    const record = await repository.insert({
      projectId: input.projectId,
      structureId: input.structureId,
      inspectorId: user.sub, // always from the verified token, never the body
      inspectionDate: input.inspectionDate,
      conditionRating: input.conditionRating,
      findings: input.findings || [],
      priority: calculatePriority(input.conditionRating, input.findings),
      status: 'draft',
    });

    await publisher.publish({ type: 'InspectionCreated', id: record.id, priority: record.priority });
    return record;
  }
```

4.2 System Integration Summary

Integration mostly meant agreeing on three contracts. The token contract: the auth-service signs a JWT with the user ID, role, and issuer, and the inspection-service checks all three before any business logic runs (Figure 4.4). The repository contract: the PostgreSQL repository has the exact same methods as the in-memory one from Unit 5, so the service code didn't change at all, and it saves an inspection with its findings in one transaction (Figure 4.5). The event contract: each change is written to the outbox table (Figure 4.6). Finally, server.js picks the repository from an environment variable and passes everything into the service (Figure 4.7). That dependency injection is why the Unit 5 unit tests kept working after a real database was added. Table 4.1 lists the tagged releases.

Figure 4.4

Token and Role Check in the inspection-service

```
Create Jira Issue
// Checks the JWT issued by auth-service and makes sure the user has an allowed role.
function requireRole(...allowed) {
  return (req, res, next) => {
    const token = (req.headers.authorization || '').replace('Bearer ', '');
    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET, {
        algorithms: ['HS256'],
        Create Jira Issue
        issuer: 'oms-auth-service',
      });
      if (!allowed.includes(payload.role)) return res.status(403).json({ error: 'Forbidden' });
      req.user = payload;
      next();
    } catch {
      res.status(401).json({ error: 'Session expired or invalid' });
    }
  };
}

module.exports = { requireRole };
```

Figure 4.5

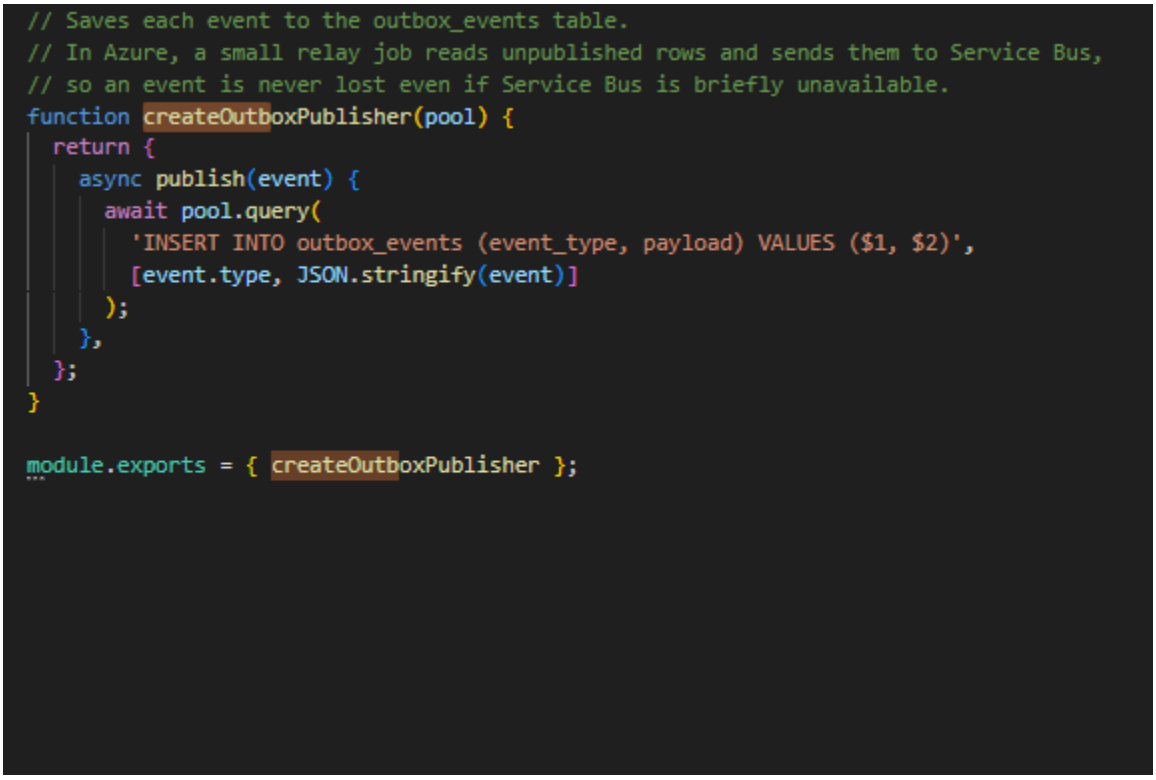
Transaction in the PostgreSQL Repository



```
return {
  async insert(data) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      const { rows } = await client.query(
        `INSERT INTO inspections
           (project_id, structure_id, inspector_id, inspection_date, condition_rating, priority, status)
         VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *`,
        [data.projectId, data.structureId, data.inspectorId, data.inspectionDate,
          data.conditionRating, data.priority, data.status]
      );
      for (const f of data.findings) {
        await client.query(
          'INSERT INTO findings (inspection_id, element, severity) VALUES ($1, $2, $3)',
          [rows[0].id, f.element, f.severity]
        );
      }
      await client.query('COMMIT');
      return toRecord(rows[0], data.findings.map(({ element, severity }) => ({ element, severity })));
    } catch (err) {
      await client.query('ROLLBACK');
      throw err;
    } finally {
      client.release();
    }
  },
```

Figure 4.6

The Outbox Publisher



```
// Saves each event to the outbox_events table.
// In Azure, a small relay job reads unpublished rows and sends them to Service Bus,
// so an event is never lost even if Service Bus is briefly unavailable.
function createOutboxPublisher(pool) {
  return {
    async publish(event) {
      await pool.query(
        'INSERT INTO outbox_events (event_type, payload) VALUES ($1, $2)',
        [event.type, JSON.stringify(event)]
      );
    },
  };
}

module.exports = { createOutboxPublisher };
```

Figure 4.7

Wiring the Modules Together in server.js

```
let repository;
let publisher;
let pool;

if (config.repoDriver === 'postgres') {
  pool = createPool(config);
  repository = createPostgresRepository(pool);
  publisher = createOutboxPublisher(pool);
} else {
  repository = createInMemoryRepository();
  publisher = { publish: async (event) => console.log('[event]', event.type) };
}

const service = createInspectionService({ repository, publisher });
const app = createApp(service, { pool });

app.listen(config.port, () => {
  console.log(`inspection-service listening on ${config.port} (repo: ${config.repoDriver})`);
});
```

Table 4.1

Tagged Milestones

Tag	Milestone	Main content
v0.1.0	Initial development	Validation, priority, and workflow logic
v0.2.0	Testing complete	32 unit tests, three bugs fixed, CI workflow
v1.0.0	Module release	Version bump and changelog
v1.1.0	Full system integration	auth-service, gateway, UI, PostgreSQL, outbox, Docker, load tests
v1.1.1	System testing fixes	403 status fix, acceptance and stability tests

4.3 Output and Screenshots

I created demo data through the same public API the UI uses (Figure 4.8), which also checked the whole chain from login to storage. The login screen asks for a work email and password (Figure 4.9). Figure 4.10 shows the inspector dashboard after saving structure with a rating of 4 (Satisfactory) and one critical finding. The system set it to IMMEDIATE and put it at

the top, above records with worse overall ratings. Engineers get Approve and Send back buttons (Figure 4.11), and managers get a read-only view (Figure 4.12). Figure 4.13 shows seven requests through the gateway, including a rejected record with three problems, a saved record with priority HIGH, a blocked attempt to approve a draft (409), and the three outbox events for one record. More screenshots are in Appendix C (Figures C.1 to C.3).

Figure 4.8

Demo Data Created Through the API

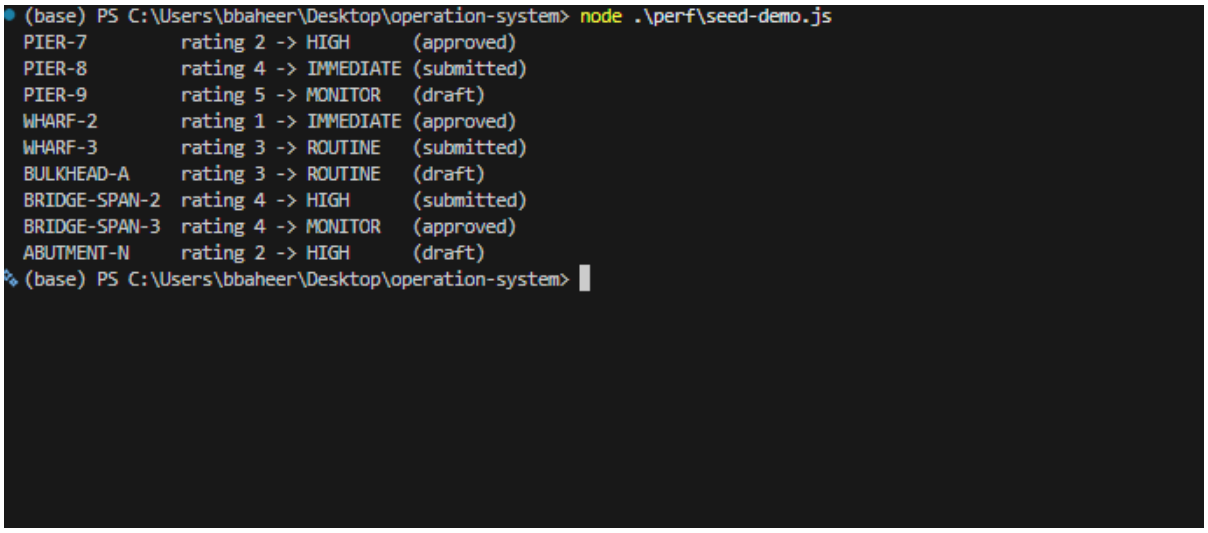


Figure 4.9

Login Screen

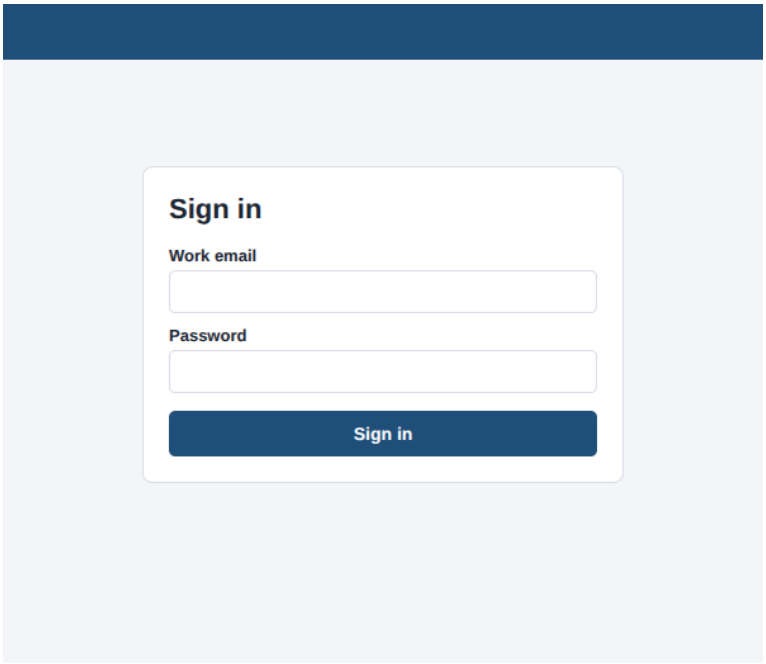


Figure 4.10

Inspector Dashboard After Saving an Inspection With a Critical Finding

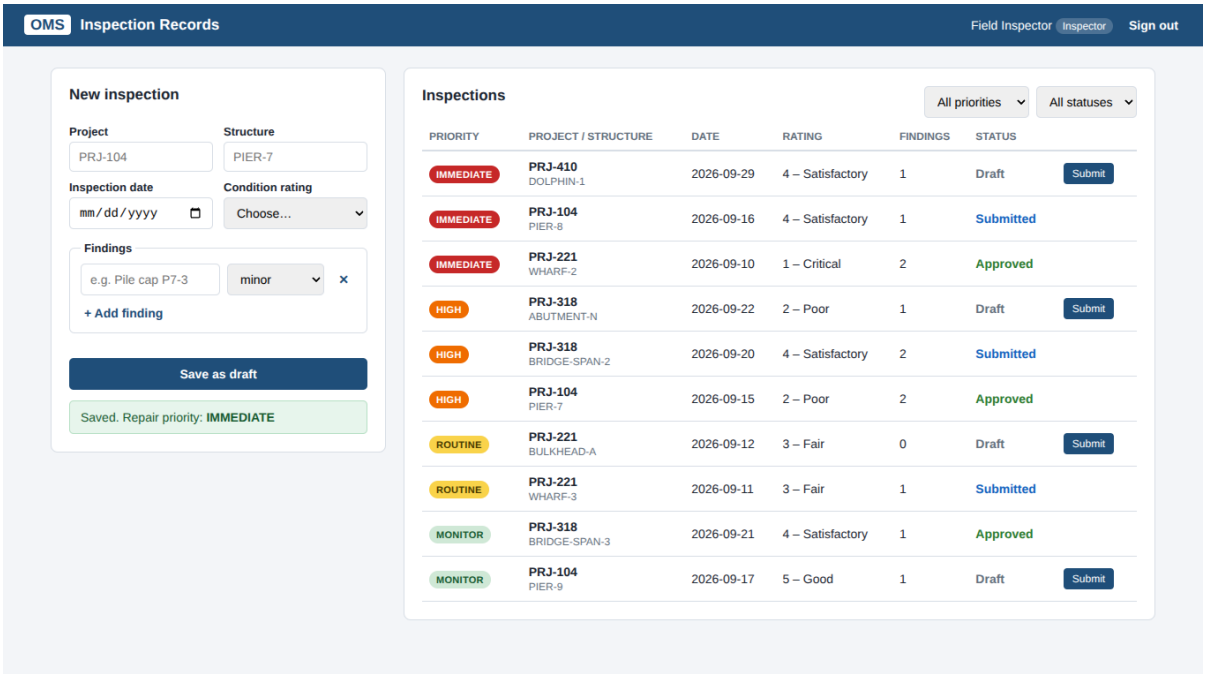


Figure 4.11

Engineer Dashboard With Approve and Send Back Actions

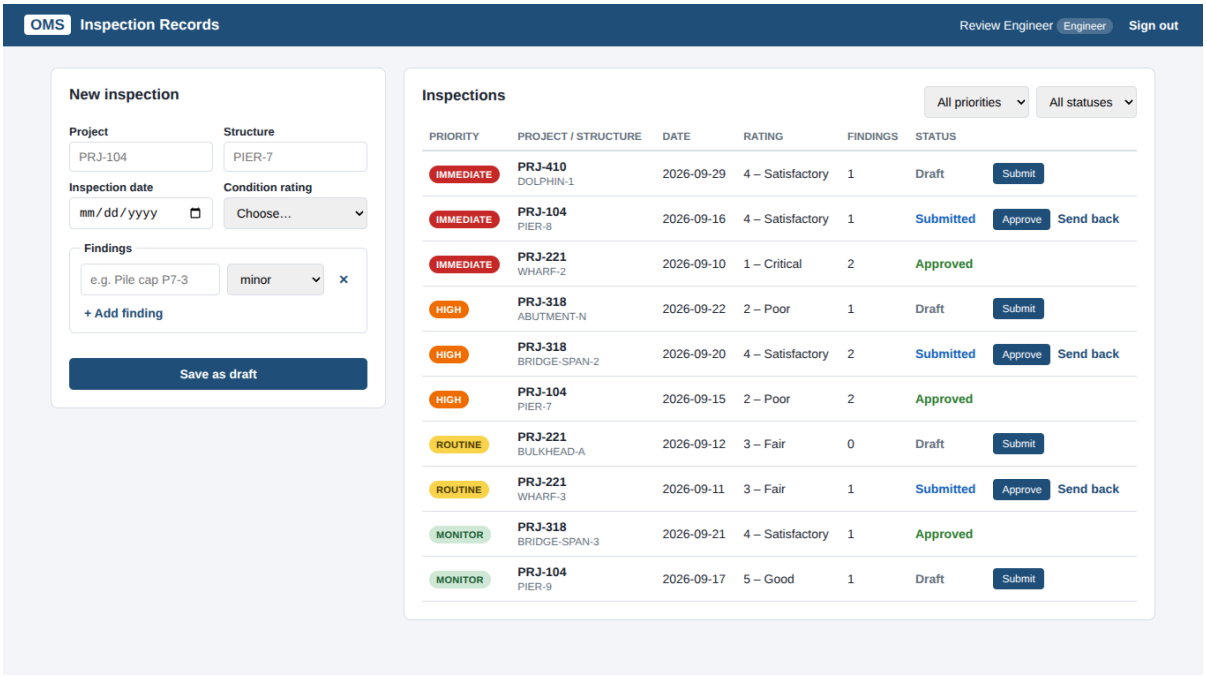


Figure 4.12

Read-Only Manager View

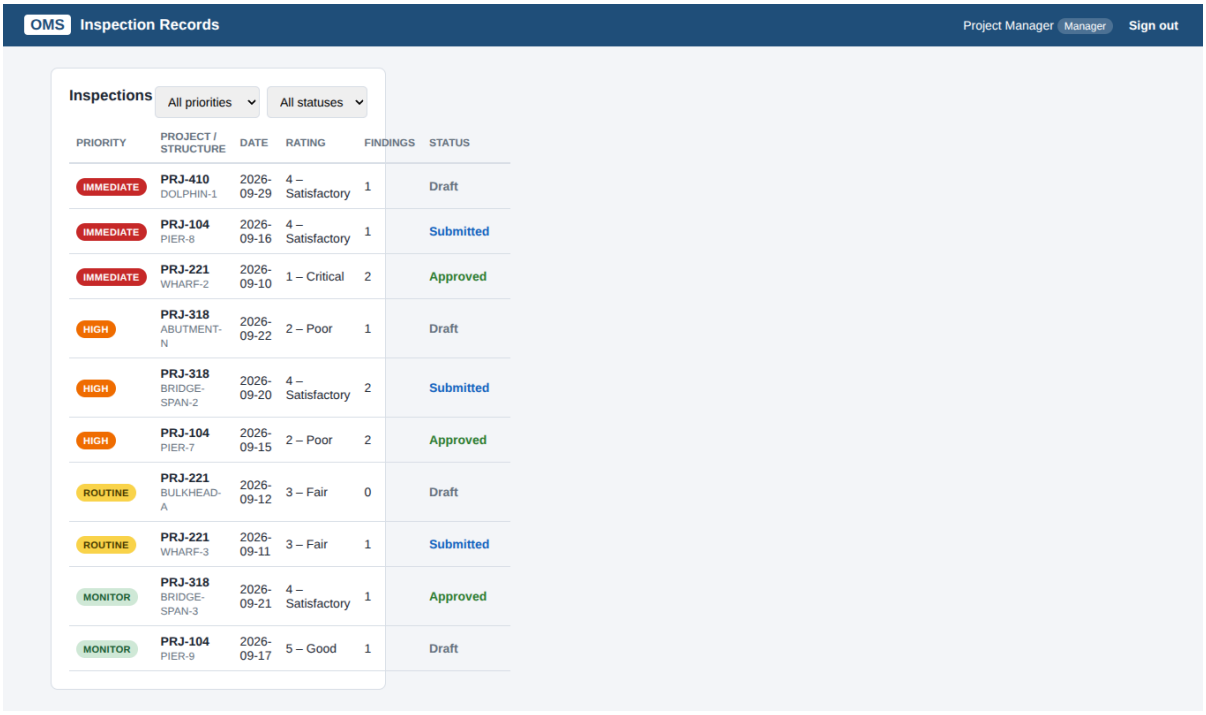
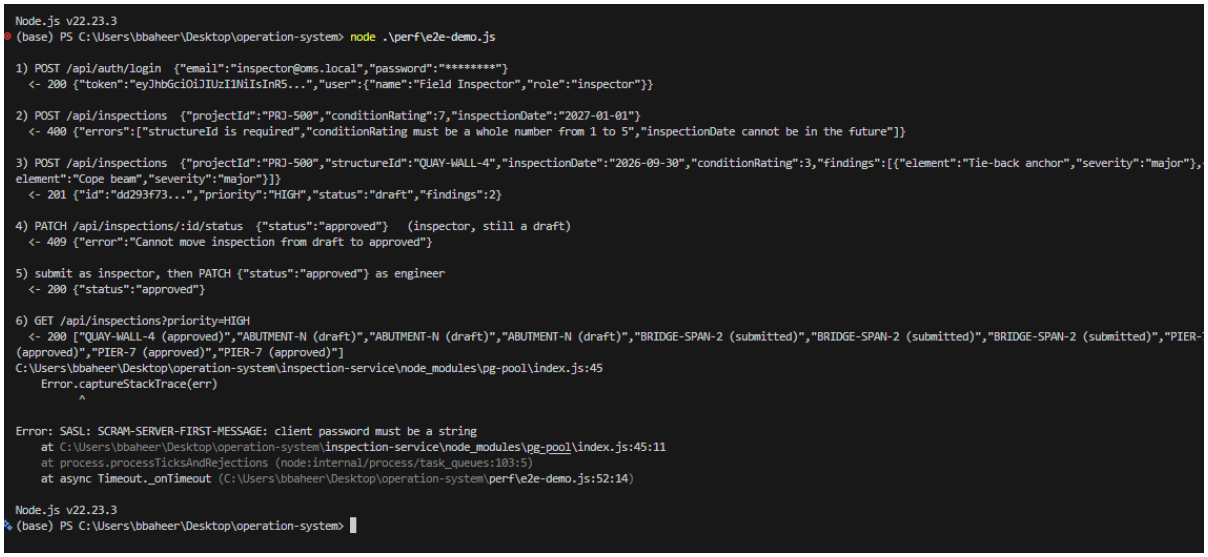


Figure 4.13

End-to-End Input and Output Through the Gateway



4.4 Testing Results

In Unit 7 I re-ran every automated suite on a clean checkout, brought up the whole system, and added acceptance and system tests. The test environment was Ubuntu 24.04 with 1 vCPU, 4 GB of RAM, Node.js 22, and PostgreSQL 16.

4.4.1 Unit, API, and Integration Tests

All 46 automated tests passed after the fix (Table 4.2, Figures 4.14 and 4.15). Line coverage for the inspection-service at v1.1.0 was 89% (Figure 4.16).

Table 4.2

Automated Test Results (v1.1.1)

Suite	Type	Tests	Result
inspectionRules.test.js	White-box unit	19	Passed
inspectionService.test.js	Unit with fakes	7	Passed
inspections.api.test.js	Black-box API	10	Passed
postgresRepository.int.test.js	Integration (PostgreSQL)	6	Passed
auth.api.test.js	Black-box API	4	Passed

Suite	Type	Tests	Result
Total		46	100%

Figure 4.14

Unit and API Tests After the Fix



Figure 4.15

Integration Tests Against PostgreSQL and auth-service Tests

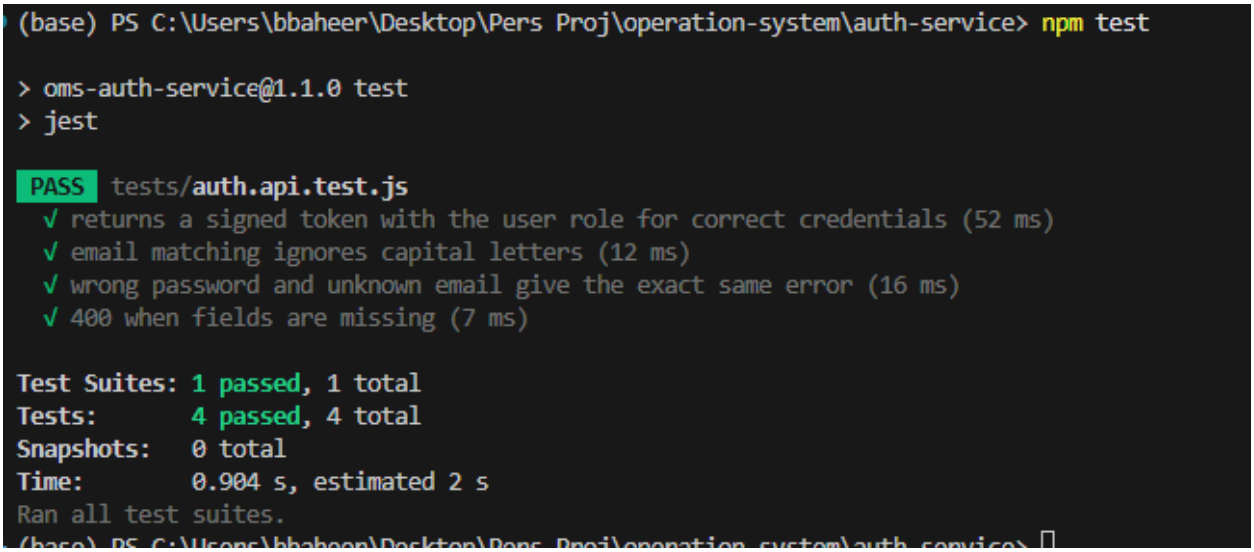
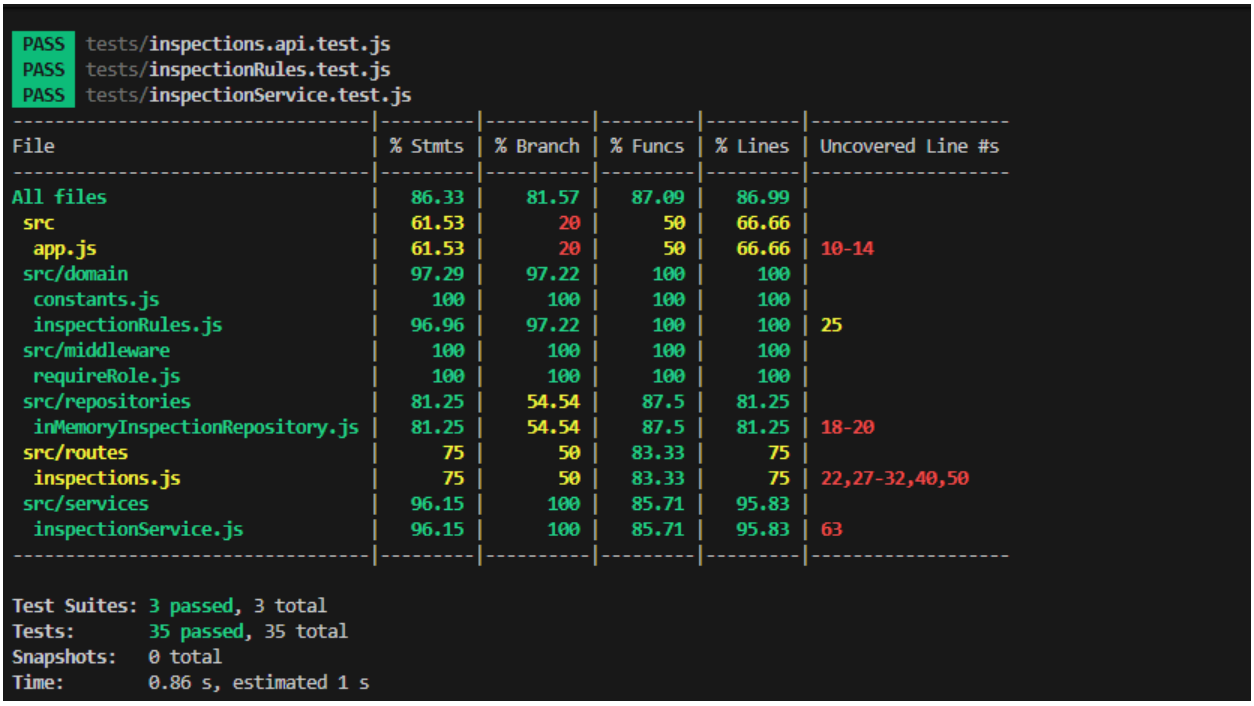


Figure 4.16

Combined Unit and Integration Coverage for the inspection-service (v1.1.0)



4.4.2 Failed Tests and How They Were Fixed

The most useful results were failures. During acceptance testing, AT-07 failed: an inspector trying to approve a submitted record got 409 Conflict instead of 403 Forbidden. The approval was still blocked, so it wasn't a security hole, but the API gave the wrong reason. The cause was that the "only engineers can approve" rule threw the same WorkflowError as an illegal status change, and every WorkflowError became a 409. My unit test only checked that some WorkflowError was thrown, so it passed. I added a PermissionError mapped to 403 and wrote an API regression test. Figure 4.17 shows that test failing on the old code (expected 403, received 409), and it passes on the fix.

Figure 4.18 shows the same kind of evidence from Unit 6. My integration test fails on the commit before the UUID fix, because PostgreSQL throws "invalid input syntax for type uuid" for an ID like "does-not-exist," which the API turned into a 500 instead of a 404. The in-memory store used in unit tests accepted any ID, so only the real database exposed it.

Figure 4.17

Failed Test Before the Fix: Expected 403, Received 409

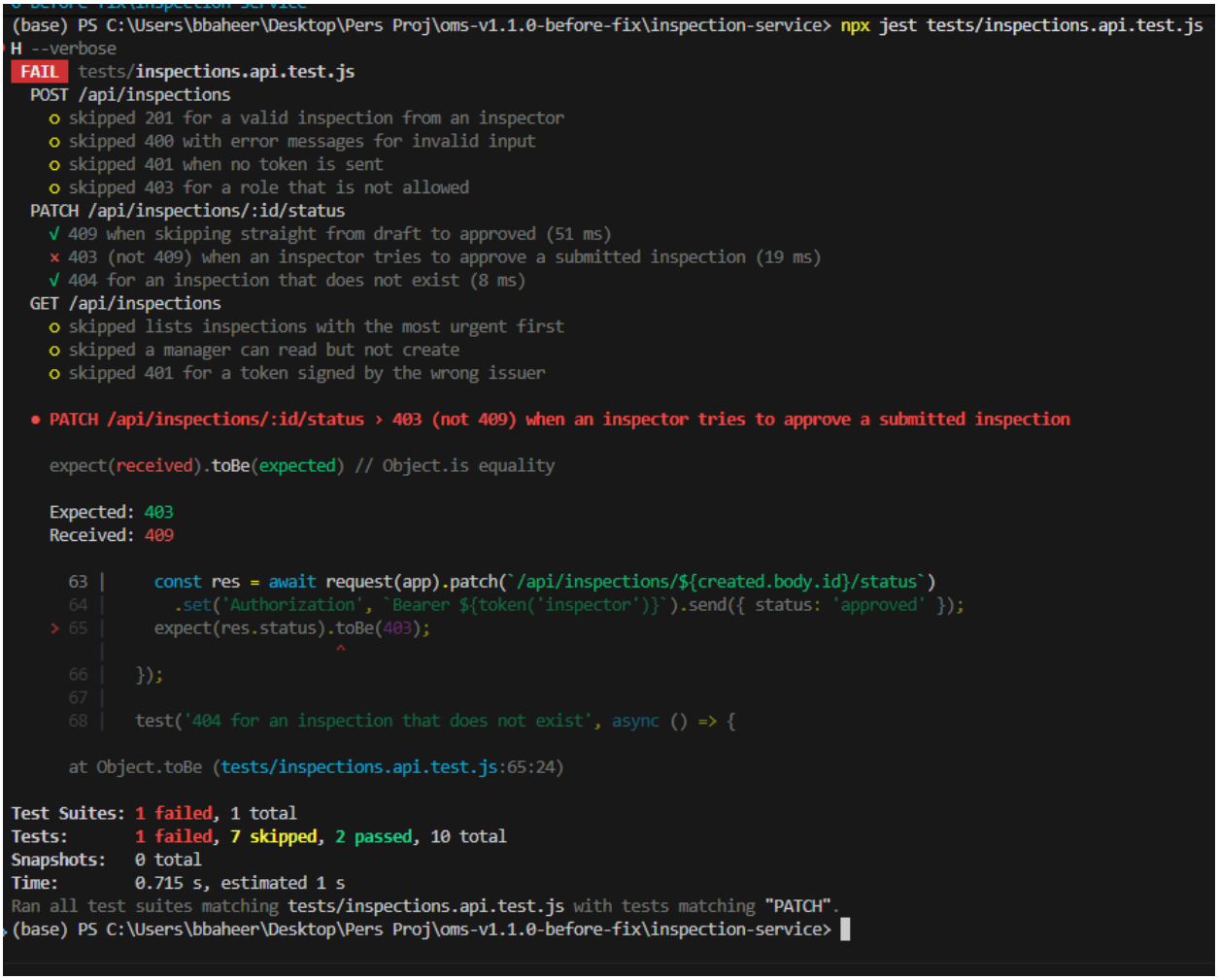


Figure 4.18

Failed Test on the Pre-Fix Commit: Non-UUID ID Crashes the Query

```
FAIL tests/integration/postgresRepository.int.test.js
  ✓ saves an inspection and its findings in one transaction (59 ms)
  ✓ writes an InspectionCreated event to the outbox table (7 ms)
  ✓ moves through the full review workflow and persists the status (15 ms)
  ✓ filters the list by project and priority (3 ms)
  × returns null (not a crash) for an id that is not a valid UUID (6 ms)
  ✓ the database rejects a rating outside 1-5 even if the API check is skipped (9 ms)
  ● returns null (not a crash) for an id that is not a valid UUID
    error: invalid input syntax for type uuid: "does-not-exist"
      22 | function createPostgresRepository(pool) {
      23 |   async function findById(id) {
    > 24 |     const { rows } = await pool.query('SELECT * FROM inspections WHERE id = $1', [id]);
         |                     ^
      25 |     if (!rows[0]) return null;
      26 |     const f = await pool.query(
      27 |       'SELECT element, severity FROM findings WHERE inspection_id = $1 ORDER BY element',
      at ../../oms/inspection-service/node_modules/pg-pool/index.js:45:11
      at Object.findById (src/repositories/postgresInspectionRepository.js:24:22)
      at Object.<anonymous> (tests/integration/postgresRepository.int.test.js:57:10)
Test Suites: 1 failed, 1 total
Tests:       1 failed, 5 passed, 6 total
```

4.4.3 Acceptance Tests and Requirement Traceability

The 12 acceptance tests run against the live system through the gateway (Figure 4.19). After the fix, 11 of 12 passed. The only failure, AT-12, checks draft editing, a feature that doesn't exist yet, so it is a known gap and not a defect. The full log with every Given, When, and Then step is in Appendix B (Figure B.1). Table 4.3 traces every requirement to its tests.

Figure 4.19

Acceptance Test Results After the Fix

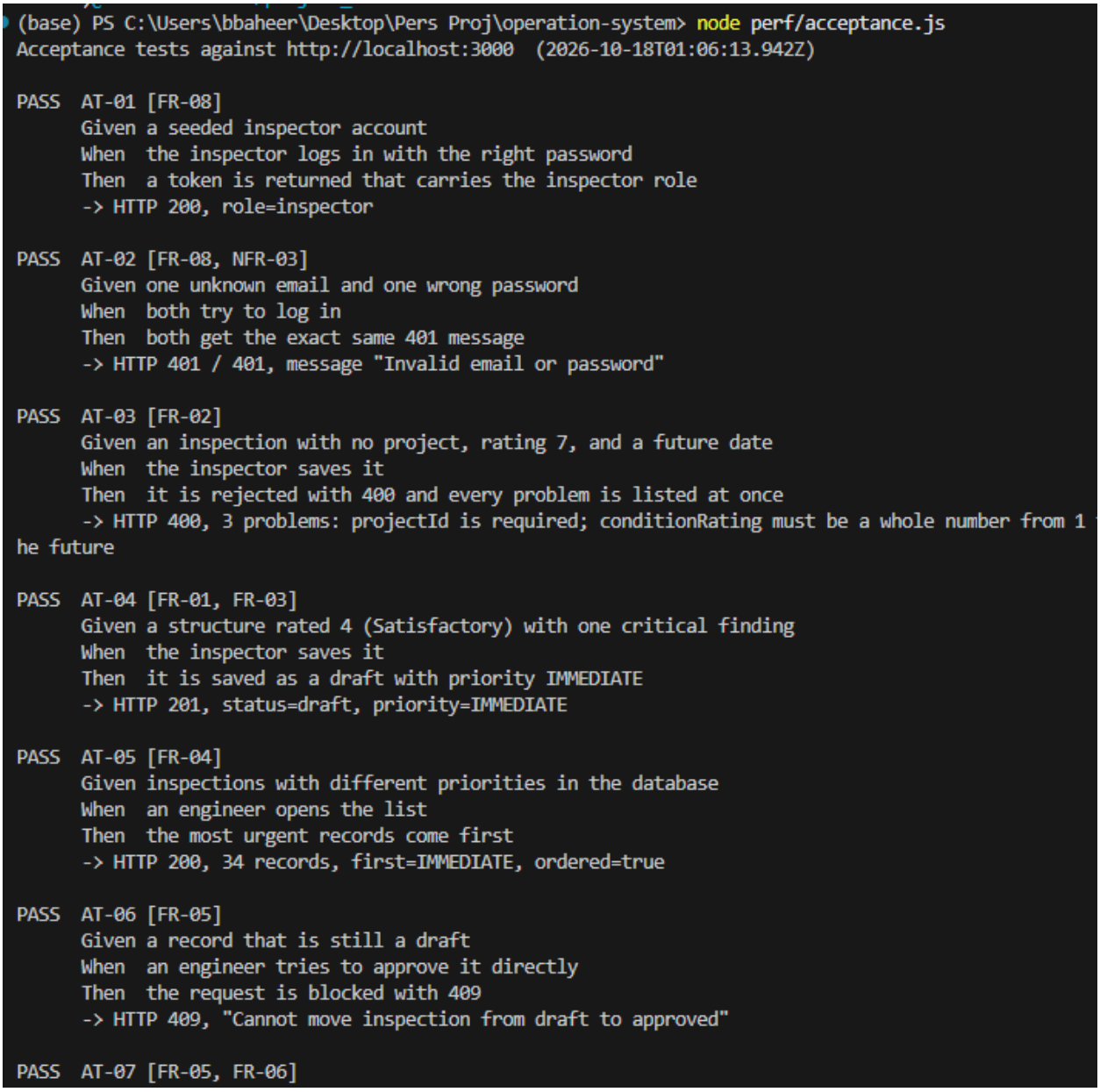


Table 4.3

Requirement Traceability and Results

Requirement	**Verified by**	**Result**
FR-01 Create inspection	AT-04, API and integration tests	Met
FR-02 Validation	AT-03, 10 unit tests	Met
FR-03 Repair priority	AT-04, 6 unit tests	Met
FR-04 Ordered, filtered list	AT-05, integration test	Met (capped at 100 rows, see L-02)
FR-05 Review workflow	AT-06, AT-07, AT-08, unit tests	Met after fix B-05
FR-06 Role-based access	AT-07, AT-09, API tests	Met after fix B-05
FR-07 Outbox events	AT-11, integration test	Met
FR-08 Login	AT-01, AT-02, auth tests	Met
FR-09 Edit draft	AT-12	Not met (planned v1.2.0)
NFR-01 Performance	Load test (Section 4.5)	Met: 13 ms read, 30 ms save
NFR-02 Reliability	Load and soak tests, row counts	Met: 0 errors, 100% consistency
NFR-03 Security	AT-02, AT-10, API tests	Met
NFR-04 Usability	Heuristic review	Met: 4.0 of 5
NFR-05 Resources	Memory sampling	Met: 120 MB peak
NFR-06 Maintainability	Unit tests unchanged after adding PostgreSQL	Met
NFR-07 Retrieval	API latency; volume check	Partly met (no timed user test; 100-row cap)

4.4.4 System Tests: Stability, Consistency, and Volume

To check stability, I ran 60 seconds of mixed traffic through the gateway, four reads for every write, while sampling memory every five seconds (Figure 4.20). There were zero errors in 13,451 requests, and memory rose during warm-up and then stayed flat, which means there is no sign of a memory leak. Afterwards every one of the 2,690 saved inspections had exactly two findings and one creation event, with no orphan rows (Figure 4.21). The volume check found a real limitation: the list endpoint always returns at most 100 records and ignores paging, so 2,590 of those records couldn’t be reached from the list (Figure 4.22).

Figure 4.20

60-Second Stability Test

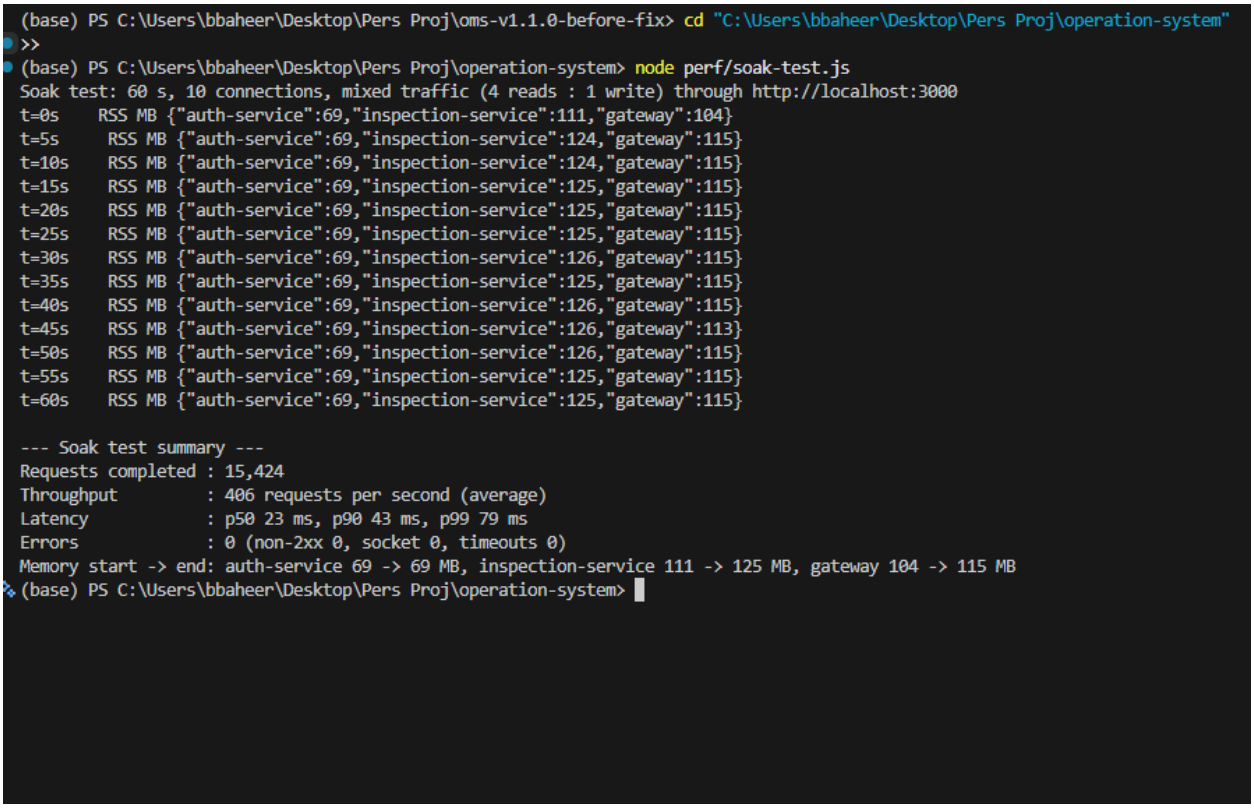


Figure 4.21

Data Consistency After the Stability Test

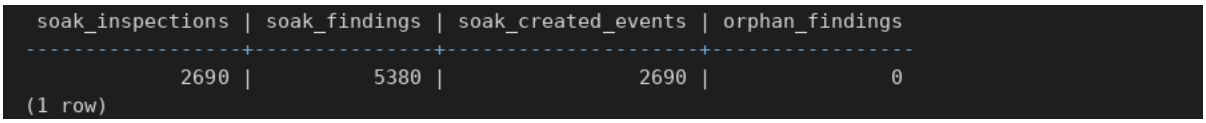
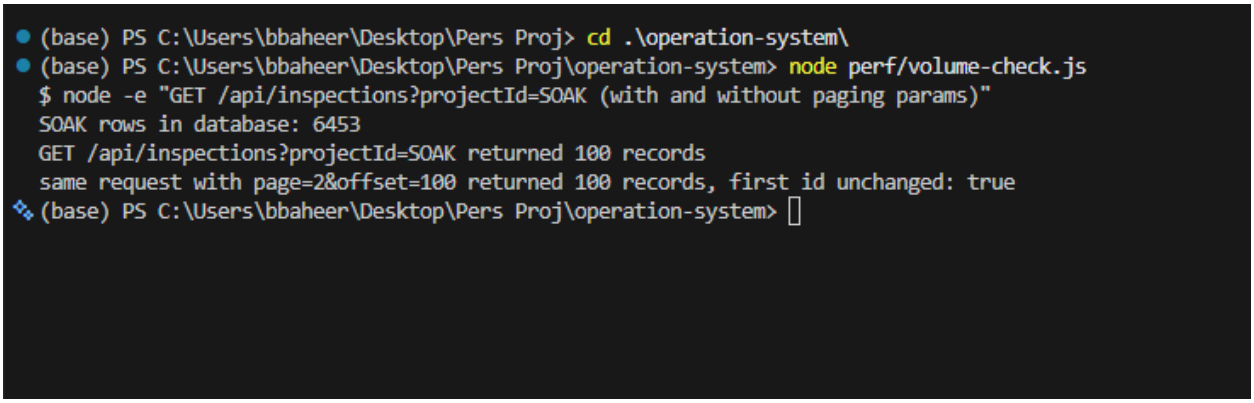


Figure 4.22

Volume Check: List Capped at 100 Records



4.4.5 Test Summary Report

Table 4.4 summarizes the Unit 7 testing as a formal test summary report. The exit criteria were set before testing, because "system testing is complete when your exit criteria are met, not when you run out of time" (BrowserStack, 2025). All of them were met. The one failed acceptance test is a feature that was never in scope for this release, and it is tracked as L-01 instead of being hidden.

Table 4.4

Test Summary Report

Item	**Details**
Objectives	Verify every functional requirement, and check that the system stays fast, stable, and consistent under load
Scope	In: auth-service, inspection-service, gateway, PostgreSQL, REST API. Out: UI layout (covered by the usability review) and Azure deployment
Environment	Windows, 24.04, 1 vCPU, 4 GB RAM, Node.js 22, PostgreSQL 16, October 17, 2026
Procedure	Unit and API tests; integration tests on a freshly migrated database; start and seed the full system; acceptance tests; stability, consistency, and volume checks; fix defects; rerun everything as regression
Exit criteria	All automated tests pass, no open defect, every requirement traced to at least one test
Results	46 of 46 automated tests, 11 of 12 acceptance tests, 0 errors in 13,451 stability requests, 100% data consistency, 1 bug fixed, 2 new limitations documented

4.5 Performance Evaluation

In Unit 6 I loaded every scenario with autocannon. Table 4.5 lists the exact conditions and Table 4.6 gives the results, since a number like “390 requests per second” means nothing without them. Everything shared one CPU, so the results are a lower bound and are best used to compare scenarios.

Table 4.5

Benchmarking Conditions

Item	**Setting**
Hardware and OS	One shared virtual CPU, 4 GB RAM, Linux
Software	Node.js 20, PostgreSQL 16, services run directly with node
Load shape	10 concurrent connections, 15 seconds per scenario, no warm-up
Units	Throughput in requests per second (req/s); latency in ms at p50, p90, p99
Raw data	perf/results/results.json

Table 4.6

Load Test Results (10 Connections, 15 Seconds per Scenario)

Scenario	**Requests**	**Throughput (req/s)**	**p50 (ms)**	**p90 (ms)**	**p99 (ms)**
Gateway health check	54,885	3,659	1	3	23
List inspections (via gateway)	5,847	390	21	44	80
List inspections (direct)	13,518	901	9	13	40
Get one inspection	9,356	624	13	22	47
Create inspection	4,246	283	30	54	91
Login (bcrypt cost 10)	161	11	823	1,229	2,051

Note. Total requests: 88,013. Error rate: 0% in every scenario.

Figure 4.23 shows the saved results file from the run, and Figures 4.24 and 4.25 chart latency and throughput for every scenario.

Figure 4.23

Load Test Summary From perf/results/results.json

autocannon load test: 10 concurrent connections, 15 s per scenario (run on 2026-10-01)

Scenario	Requests completed	Throughput (req/s)	p50 (ms)	p90 (ms)	p99 (ms)	Error rate
Gateway health check	54,885	3,659	1	3	23	0%
List inspections (via gateway)	5,847	390	21	44	80	0%
List inspections (direct to service)	13,518	901	9	13	40	0%
Get one inspection	9,356	624	13	22	47	0%
Create inspection (write + outbox)	4,246	283	30	54	91	0%
Login (bcrypt cost 10)	161	11	823	1229	2051	0%

Total requests: 88,013 | Idle memory (RSS): gateway 74 MB, auth-service 63.4 MB, inspection-service 62.1 MB | Source: perf/results/results.json

Figure 4.24

Response Latency by Scenario (ms)

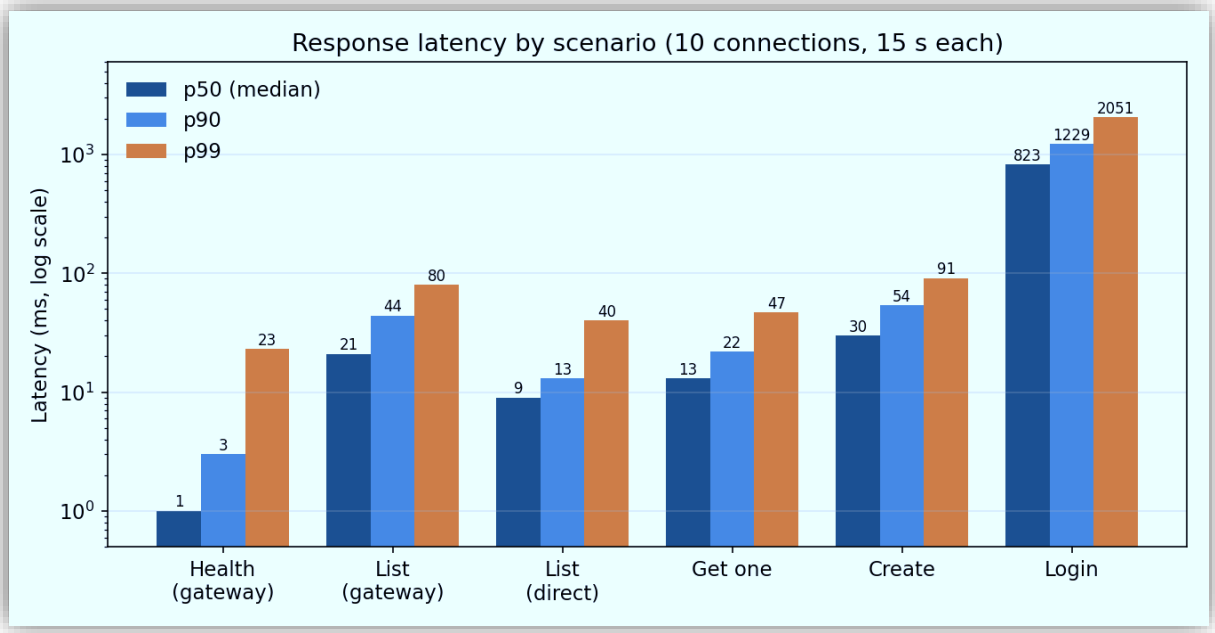
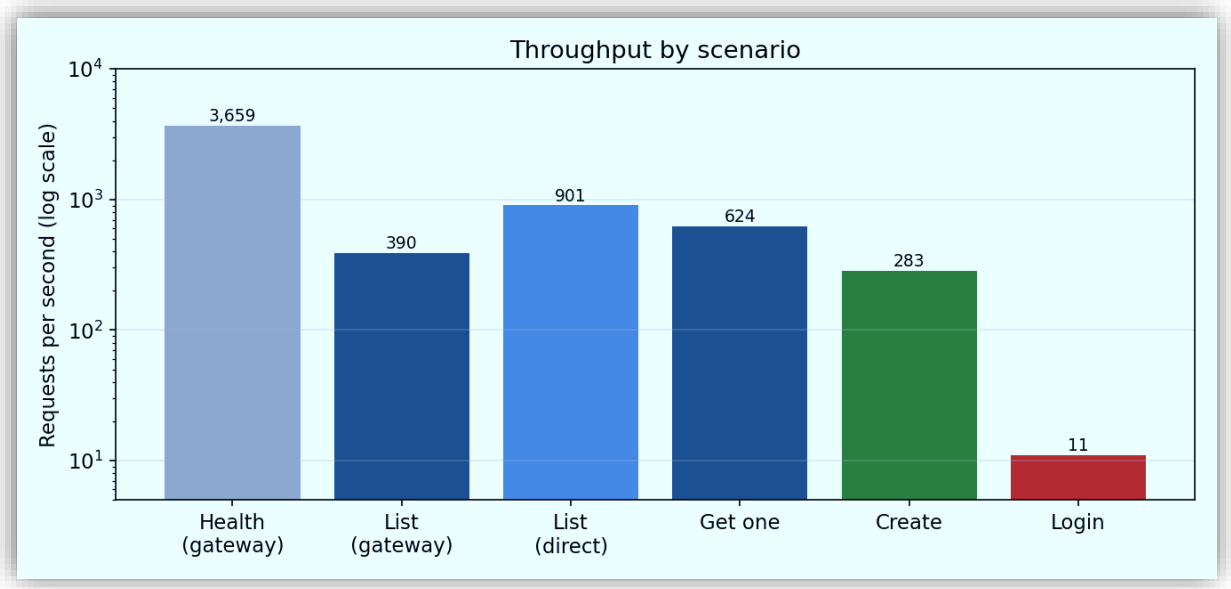


Figure 4.25

Throughput by Scenario (Requests per Second)



Memory was measured as resident set size for each service. Each one idled between 62 and 74 MB and peaked at about 120 MB under load (Figure 4.26). After the Unit 6 write test the database held 4,256 inspections (4,246 from the test plus 10 demo records), 8,512 findings, and 4,256 events, which is another 100% match (Table 4.7).

Figure 4.26

Memory Footprint per Service(MB)

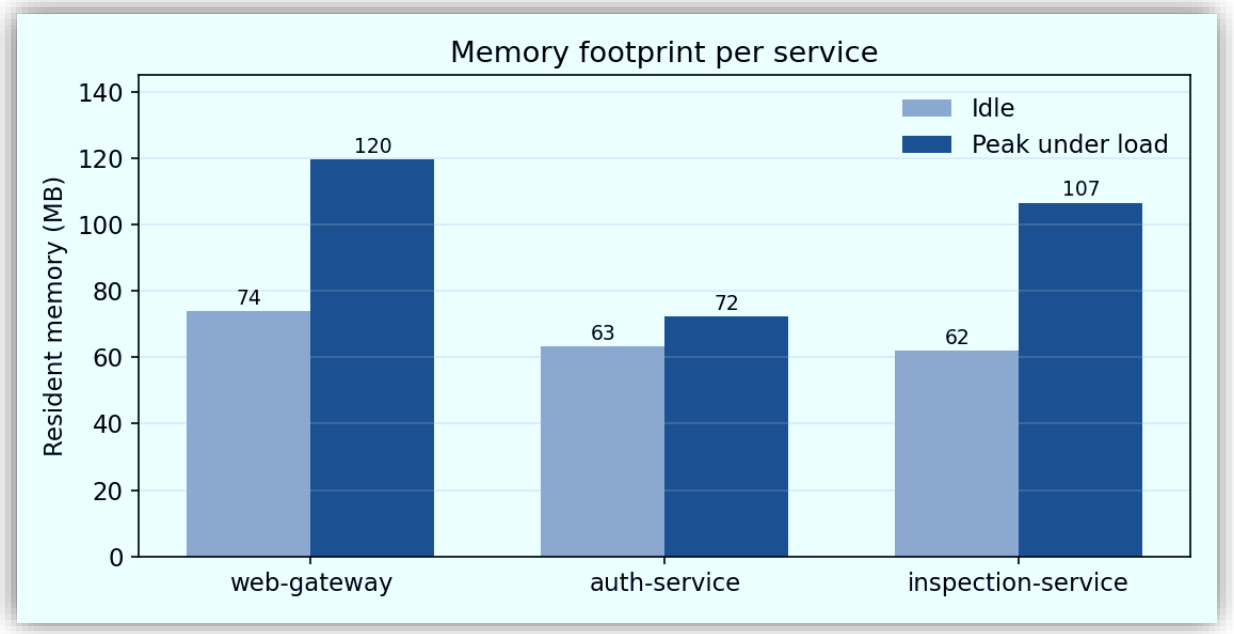


Table 4.7

Data Consistency Checks

Test	Inspections	Findings	Creation events
Unit 6 write load test	4,256	8,512	4,256
Unit 7 stability test (project SOAK)	2,690	5,380	2,690

4.6 Usability Evaluation

For the qualitative metric I scored the interface on six of Nielsen's (1994) heuristics from 1 (poor) to 5 (excellent). The biggest problem was the error messages (Figure 4.27). They came straight from the API, so an inspector saw text like "findings[0].element is required," and the empty finding row that loads with the form caused an error even when there were no findings. I rewrote the messages in plain language with examples, outlined the bad fields in red, and ignored blank rows (Figure 4.28). The average rose from 3.2 to 4.0 (Table 4.8, Figure 4.29).

Figure 4.27

Validation Messages Before the Fix

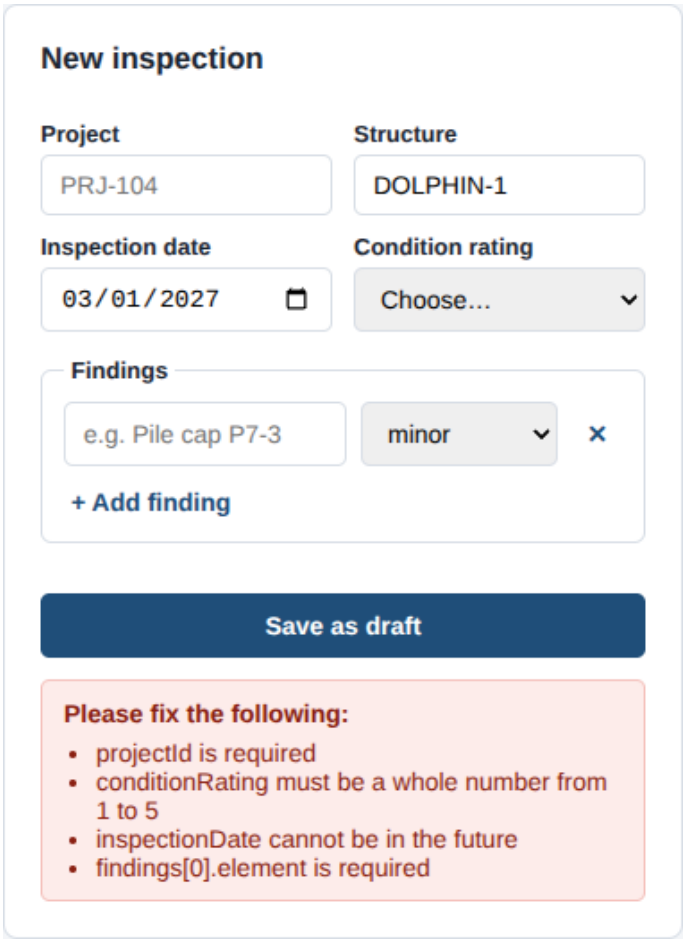


Figure 4.28

Validation Messages After the Fix

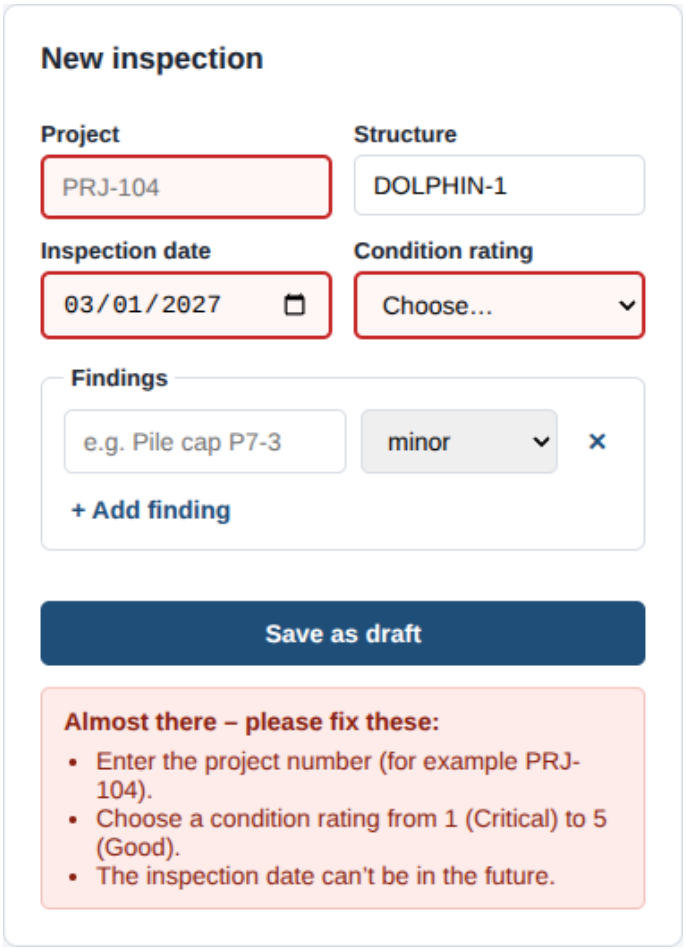


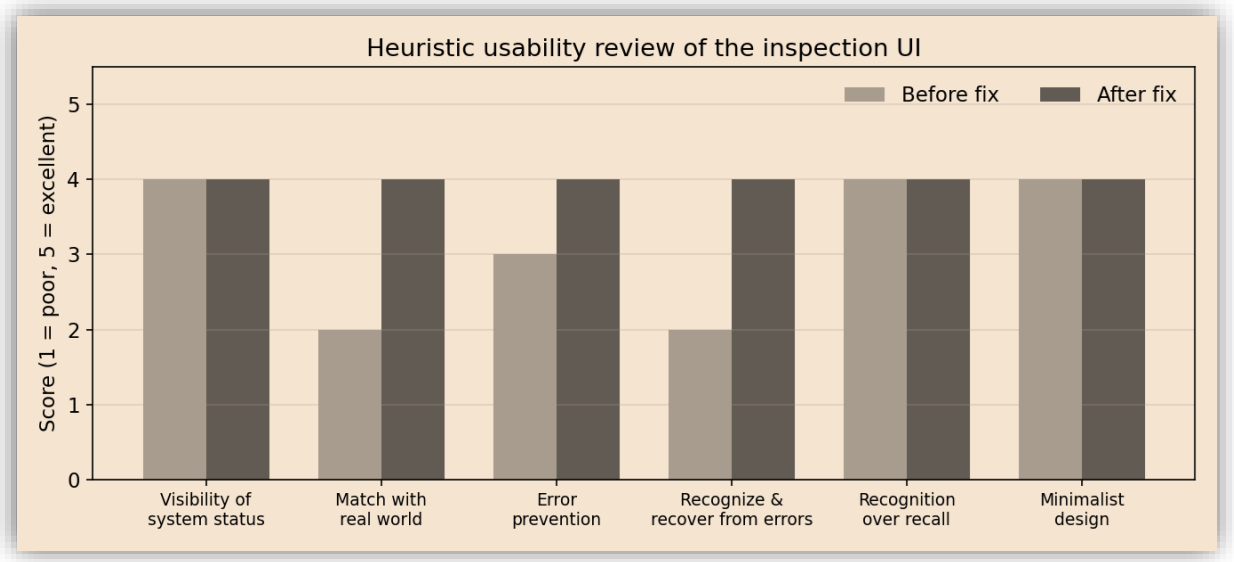
Table 4.8

Heuristic Usability Review

Heuristic	**Before**	**After**
Visibility of system status	4	4
Match between system and real world	2	4
Error prevention	3	4
Recognize, diagnose, recover from errors	2	4
Recognition rather than recall	4	4
Aesthetic and minimalist design	4	4
Average	3.2	4.0

Figure 4.29

Usability Scores Before and After the Fix



4.7 Summary of Bugs and Limitations

Table 4.9 lists everything testing found across the project. Five bugs were fixed, each with a test that now guards against it, and four limitations are documented for future releases.

Table 4.9

Bugs, Issues, and Limitations Found by Testing

Found by	**Problem**	**Solution**
Unit test	Rating 5 rejected	Fixed boundary check
Unit test	Future dates accepted; unknown status crashed	Date check; safe default
Integration test	Non-UUID ID returned 500	Bad IDs treated as not found
Usability review	Developer-style error messages	Plain-language messages
Acceptance AT-07	Inspector approval returned 409, not 403	PermissionError mapped to 403
Acceptance AT-12	Drafts cannot be edited	Edit endpoint
Volume check	List capped at 100, no paging	pagination

Found by	Problem	Solution
Load test	Login only 11 req/s	Native bcrypt, rate limiting
Load test on Windows	Load script needs ps and /proc	Run under WSL 2; make portable

Chapter 5: Discussion

5.1 Interpretation of Results

The first thing the results show is that the core design works. The priority rules gave the right answer in every test and demo, and the workflow blocked every illegal move. Getting a 409 when trying to approve a draft is not a failure. It's the system doing its job.

The second thing is that normal API calls are fast enough for this kind of tool. Reading one inspection took 13 ms at the median and saving one took 30 ms, with zero errors in 88,013 load-test requests and 13,451 stability-test requests. A team of a few dozen inspectors will never come close to 283 saves per second.

The third thing is that the gateway has a cost. Listing through the gateway was about twice as slow as calling the service directly (21 ms versus 9 ms), and throughput fell from 901 to 390 req/s. Part of that is the extra hop, but a big part is that the gateway, the service, PostgreSQL, and the load tester all shared one CPU. I still think the gateway is worth it, because it hides internal addresses and gives the browser one place to talk to.

The fourth thing is login. At 11 req/s and an 823 ms median, it is by far the slowest action. Part of that is on purpose, since bcrypt is meant to be slow (OWASP Foundation, n.d.). The bigger issue is where the work runs. bcryptjs is plain JavaScript, and Node.js runs JavaScript on one main thread, and the Node.js guide says "You should make sure you never block the Event Loop" (Node.js, n.d.). Its async compare breaks the hash into chunks, but every chunk still

runs on that one thread, so 10 logins take turns. The math fits: at 11 per second, each hash costs about 90 ms of CPU, so 10 logins waiting in line should take about 10 ÷ 11, or 0.9 seconds, and the measured average was 900 ms. People log in once per shift, so it rarely matters, but it would at the start of a busy shift.

The fifth thing is about testing itself. Every level caught something the level below could not. Unit tests found boundary bugs, the integration test found the UUID bug, the acceptance test found the wrong status code, the volume check found the 100-row cap, and the usability review found confusing messages that passed every automated test. If I had stopped at unit tests, four of those nine items would still be hidden.

5.2 Comparison With the Literature

Pallante et al. (2024) argue that a central relational database keeps infrastructure data usable, and my results agree: one PostgreSQL store with constraints and transactions stayed 100% consistent through two heavy write tests. Li et al. (2023) show coded defect data driving maintenance decisions, and my severity-coded findings drive the repair priority the same way, just at a much smaller scale.

My testing results line up with BrowserStack (2025), which describes system testing as checking that “everything works together correctly.” AT-07 only failed when the parts were tested together through HTTP. Summers (2020) notes that “software metrics can be used to control software engineering process or to predict product attributes,” and my latency, error, and memory numbers now act as a baseline for future releases. Wells (2025) and LaunchNotes (2023) recommend mixing metric types, and my project is a clear case: the numbers said the form worked, but only the heuristic review showed it was confusing.

Sumo Logic (2025) says continuous deployment works for teams that have "invested heavily in automated testing," and the 46 automated tests plus 12 acceptance tests are what make the planned Azure rollout safe. Where my work adds detail is cost. The readings present the gateway and password hashing as clear wins. My measurements show what they actually cost on small hardware, which a team should measure instead of assume.

5.3 Deployment and Configuration

I compared four deployment options (Table 5.1) and chose Docker containers on Azure Container Apps, with Azure Database for PostgreSQL, Azure Container Registry, and Key Vault. Red Hat (2025) says containers "make it easier to move applications between development, testing, and production environments," and Container Apps keeps each deployment as a revision, which supports canary releases and one-command rollback.

Table 5.1

Deployment Options Compared

Option	**Strengths**	**Weaknesses**
On-premise server	Cheap to start, full control	Manual patching, backups, scaling
Azure App Service	Simple, managed	Awkward for three private services
Azure Kubernetes Service	Where the larger OMS is heading	Heavy setup and cost for three services
Docker on Azure Container Apps	Same images as local; revisions; rollback	Needs registry, Key Vault, managed database

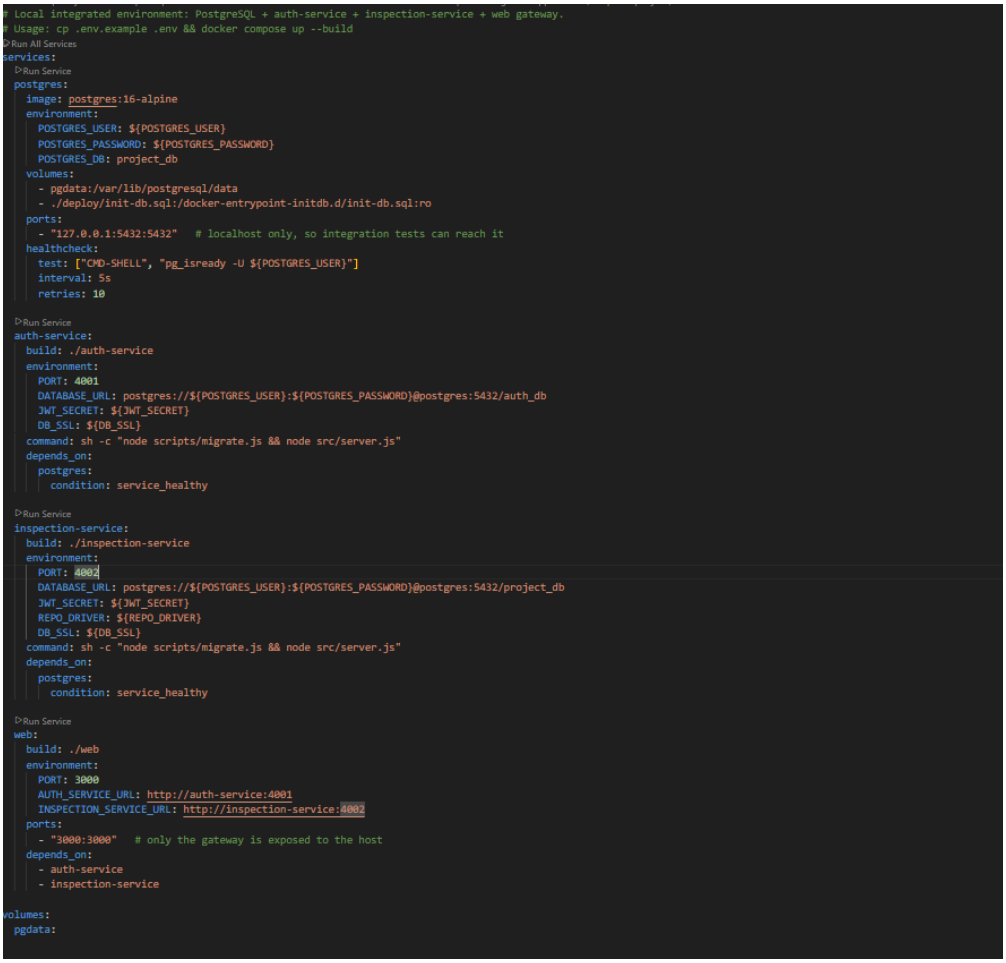
The full system runs end to end with Docker Compose (Figure 5.1), and all results in Chapter 4 came from locally integrated systems. The Azure deployment is fully written out but has not been run, mainly because the Azure free account's $200 credit only lasts 30 days, and a managed database, registry, Key Vault, and three apps start costing money as soon as they exist.

The plan is one short session to deploy, rerun the load test, capture a rollback, and delete the resource group.

Configuration follows the twelve-factor idea of keeping settings in environment variables (Wiggins, n.d.). Each service refuses to start if a required variable is missing, secrets come from Key Vault in Azure and from a .env file that Git ignores locally, and package-lock.json pins every library version (Table 5.2). Each Git tag is a baseline, and Guru99 puts it simply: “In simple words, baseline means ready for release” (Bennett, 2024). Because the image tag matches the Git tag, anyone can confirm which reviewed code is running. Appendix A has the full setup commands.

Figure 5.1

Docker Compose Configuration for the Local Integrated Environment



```
# Local integrated environment: PostgreSQL + auth-service + inspection-service + web gateway.
# Usage: cp .env.example .env && docker compose up --build
Run All Services
services:
  Run Service
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: project_db
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./deploy/init-db.sql:/docker-entrypoint-initdb.d/init-db.sql:ro
    ports:
      - "127.0.0.1:5432:5432"   # localhost only, so integration tests can reach it
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      retries: 10

  Run Service
  auth-service:
    build: ./auth-service
    environment:
      PORT: 4001
      DATABASE_URL: postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/auth_db
      JWT_SECRET: ${JWT_SECRET}
      DB_SSL: ${DB_SSL}
    command: sh -c "node scripts/migrate.js && node src/server.js"
    depends_on:
      postgres:
        condition: service_healthy

  Run Service
  inspection-service:
    build: ./inspection-service
    environment:
      PORT: 4002
      DATABASE_URL: postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/project_db
      JWT_SECRET: ${JWT_SECRET}
      REPO_DRIVER: ${REPO_DRIVER}
      DB_SSL: ${DB_SSL}
    command: sh -c "node scripts/migrate.js && node src/server.js"
    depends_on:
      postgres:
        condition: service_healthy

  Run Service
  web:
    build: ./web
    environment:
      PORT: 3000
      AUTH_SERVICE_URL: http://auth-service:4001
      INSPECTION_SERVICE_URL: http://inspection-service:4002
    ports:
      - "3000:3000"   # only the gateway is exposed to the host
    depends_on:
      - auth-service
      - inspection-service

volumes:
  pgdata:
```

Table 5.2

Environment Variables

Variable	**Used by**	**Purpose**
JWT_SECRET	auth, inspection	Signs and checks tokens (Key Vault in Azure)
DATABASE_URL	auth, inspection	PostgreSQL connection string (Key Vault in Azure)
DB_SSL	auth, inspection	false locally, true in Azure
REPO_DRIVER	inspection	postgres, or memory for unit tests
PORT	each service	web 3000, auth 4001, inspection 4002
AUTH_SERVICE_URL, INSPECTION_SERVICE_URL	web	Where the gateway forwards requests

5.4 Maintenance Plan

Thales (n.d.) describes maintenance as “the process of changing, modifying, and updating software to keep up with customer needs” and lists four types, all of which apply here. Corrective maintenance fixes faults, like the B-05 hotfix. Adaptive maintenance responds to “changing technologies as well as policies and rules” (Thales, n.d.). The most urgent example is that Node.js 20 reached end of life on April 30, 2026 (HeroDevs, 2026), so the node:20-alpine images need to move to Node.js 22, and my Unit 7 tests already passed on Node.js 22. Perfective maintenance adds features users need, like draft editing and pagination. Preventive maintenance fixes “latent faults” before they become “effective faults” (Thales, n.d.), like moving the outbox write inside the transaction.

The schedule is continuous health checks and error alerts, all tests on every push, weekly dependency and image scans, a monthly rerun of the load and stability tests against the v1.1.1 baseline, and a quarterly usability check and backup restore drill. Bug fixes go on a hotfix branch from main with a failing test written first, and releases follow MAJOR.MINOR.PATCH numbering. If a release goes wrong, git revert “creates a new commit” that undoes the change without rewriting history (Davies, 2024). The main post-deployment risks are access-control regressions, vulnerable dependencies, lost outbox events, login slowdowns, and usability gaps. Each one has a test or monitor that detects it, and a rollback or scaling contingency.

Table 5.4 lists the main post-deployment risks with how each one is prevented and what happens if it occurs anyway. Several of them come straight from bugs and limitations this project already hit, which makes them realistic rather than guesses.

Table 5.4

Post-Deployment Risks and Mitigation

Risk	**Mitigation**	**Contingency**
Broken access control comes back, like B-05	Acceptance tests AT-07, AT-09, and AT-10 run on every push and block merges	Hotfix branch, or roll back to the previous revision
Vulnerable library, end-of-life runtime, or leaked JWT secret	Weekly npm audit and image scans; move to Node.js 22; secrets only in Key Vault	Rotate JWT_SECRET, which safely logs everyone out
Event lost between saving and the outbox write	Move the outbox insert into the same transaction; monthly consistency query	Rebuild missing events from the inspections table
Login slows down at the start of a shift	Native bcrypt, more CPU per container, login rate limiting	Scale out more auth-service replicas
Inspectors cannot fix drafts or reach records past 100	Draft editing and pagination in v1.2.0	Engineer sends the record back; use filters
Database failure or data loss	Managed backups with point-in-time restore; quarterly restore drill	Restore to a new server
Cloud costs run over	Budget alerts, minimum replicas, test environment scales to zero	Delete the test resource group

5.5 Limitations

Test hardware. All performance numbers came from one shared CPU running everything, so they compare scenarios well but don't predict production speed. The Unit 7 tests also ran on Node.js 22 while the project still declares Node.js 20.

No cloud deployment yet. Because of the credit limit, there are no Azure measurements. The deployment section is a plan, not a result.

Usability evidence. The usability scores come from my own heuristic review, not from real field inspectors, and I haven't done the timed user test from objective O3. A developer reviewing his own interface will miss things.

Functional gaps. Drafts can't be edited, the list stops at 100 records, and the relay that forwards outbox events to Service Bus isn't written yet. The outbox event is also written right

after the transaction instead of inside it, so a crash at the wrong moment could lose an event (Richardson, n.d.).

Scope. The module covers one client's data. Multi-client separation, document links, and export from the Unit 3 plan are not built yet.

5.6 Evaluation Against the Objectives

Table 5.3 checks each objective from Chapter 1 against the evidence. Five are met and one is partly met, because the timed user test still needs to happen.

Table 5.3

Objectives Versus Results

Result	Status
Five functional endpoints plus health checks, three roles, 46 of 46 automated tests passing	Met (cloud deployment pending)
Worst-condition-first priority verified by unit and acceptance tests	Met
Filters for project, priority, and status; reads in 13 ms; no timed user test yet	Partly met
Server-side role checks, signed expiring tokens, bcrypt, synthetic data, AT-10 passed	Met
0 errors in over 100,000 requests, 120 MB peak memory, 100% consistency	Met
Reproducible setup steps, deployment guide, maintenance plan	Met

Chapter 6: Conclusion and Future Work

6.1 Summary of Main Findings

This capstone set out to fix scattered inspection data with a cloud-ready module that stores inspection records in one place, ranks them by repair urgency, and makes sure every record is reviewed. It grew from a few tested functions into a working system with a UI, a gateway, an authentication service, an inspection service, PostgreSQL storage, an event outbox,

Docker packaging, and a deployment and maintenance plan. The algorithm worked as designed, all 46 automated tests and 11 of 12 acceptance tests passed, and the system stayed fast, stable, and fully consistent under load. Testing found and fixed five bugs and documented four limitations.

6.2 Answers to the Research Questions

For the first question, the research says to keep a structured relational database at the center and to code condition data consistently so it can drive decisions, and my results back that up. For the second, a minimal practical module turned out to be three small services with clear contracts, one database, and a simple priority rule, rather than a big platform. For the third, building security and testing in from the start meant server-side checks on every request, IDs taken only from tokens, synthetic data, and tests at every level, which is how B-05 was caught before any user saw it.

6.3 Contributions

For my team at work, the module is a realistic starting point for replacing spreadsheets with one system that shows engineers the most urgent problems first, and its repository contract, token check, and outbox can be reused in other OMS services. For other developers, it is a small, complete example of taking tested logic into an integrated, measured system without rewriting it. For the field, it adds real numbers to things that are usually assumed, like what a gateway and a pure JavaScript password hash cost on small hardware, and it shows that a mid-size firm can apply the same practices as large agencies without their budget.

6.4 Lessons Learned and Challenges

The biggest lesson was that passing tests isn’t the same as being done. My form passed every test and still confused users, and a unit test passed while the real HTTP response was

wrong. The second lesson was that good structure early pays off later. Because rules and storage were separated in Unit 4, adding PostgreSQL in Unit 6 didn't change a single line of business logic. A third lesson was to write down exactly how a benchmark was run. One challenge was working on Windows: line endings made Git show every file as changed, and the load script failed because it depends on Linux tools. Another was scope control, since the Unit 3 plan was bigger than one part-time developer could build, and cutting it down honestly was the right call.

6.5 Future Work

The next release, v1.2.0, will add draft editing and cursor pagination, move the outbox write inside the transaction, add the Service Bus relay, switch to native bcrypt with login rate limiting, move to Node.js 22, and make the load script cross-platform. After that I plan to deploy to Azure in a short, credit-friendly session or to my team's test environment, rerun the load test on real cloud hardware, and run a timed usability study with three to five field inspectors. Longer term, offline drafts for inspectors working under wharves with no signal, multi-client separation, and document links would bring the module closer to the full Unit 3 vision.

References

Amazon Web Services. (n.d.). What is unit testing? https://aws.amazon.com/what-is/unit-testing/

Association for Computing Machinery. (2018). ACM code of ethics and professional conduct. https://www.acm.org/code-of-ethics

Bennett, L. (2024, August 13). Software configuration management in software engineering. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

Bit.ai Editorial Team. (2025, August 24). Create a software design document easily: Steps & template inside. Bit.ai Blog. https://blog.bit.ai/software-design-document/

BrowserStack. (2025, May 29). What is system testing? (Examples, use cases, types). https://www.browserstack.com/guide/what-is-system-testing

Davies, S. (2024). Blueprints: Creating, describing, and implementing designs for larger-scale software projects (Version 2.5). University of Mary Washington. https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis

GeeksforGeeks. (2025, July 22). Unit testing – Software testing. https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/

HeroDevs. (2026, April 9). Node.js v20 goes EOL April 30, and your cloud provider is pulling the plug the next day. https://www.herodevs.com/blog-posts/node-js-v20-goes-eol-april-30----and-your-cloud-provider-is-pulling-the-plug-the-next-day

Hrupa, A. (2024, March 19). What is system testing in software testing. TestFort. https://testfort.com/blog/what-is-system-testing-in-software-testing

Jeon, C.-H., Nguyen, D.-C., Roh, G., & Shim, C.-S. (2023). Development of BrIM-based bridge maintenance system for existing bridges. Buildings, 13(9), Article 2332. https://doi.org/10.3390/buildings13092332

LaunchNotes. (2023, September 1). Qualitative vs quantitative metrics: A comprehensive comparison. https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison

Letaw, L. (2024). Handbook of software engineering methods. Oregon State University. https://open.oregonstate.education/setextbook/

Li, S., Zhang, Z., Lin, D., Zhang, T., & Han, L. (2023). Development of a BIM-based bridge maintenance system (BMS) for managing defect data. Scientific Reports, 13, Article 846. https://doi.org/10.1038/s41598-023-27924-6

National Institute of Standards and Technology. (2020). Security and privacy controls for information systems and organizations (NIST Special Publication 800-53, Rev. 5). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-53r5

Nielsen, J. (1994). 10 usability heuristics for user interface design. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Node.js. (n.d.). Don’t block the event loop (or the worker pool). https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

Oguz, A. (2025). Project management (2nd ed.). MSL Academic Endeavors. https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/

OWASP Foundation. (2021). OWASP top 10: 2021. https://owasp.org/Top10/

OWASP Foundation. (n.d.). Password storage cheat sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Pallante, L., Meriggi, P., D'Amico, F., Gagliardi, V., Napolitano, A., Paolacci, F., Quinci, G., Lorello, M., & de Felice, G. (2024). An integrated data-driven system for digital bridge management. Buildings, 14(1), Article 253. https://doi.org/10.3390/buildings14010253

Red Hat. (2025, April 29). What is DevOps? https://www.redhat.com/en/topics/devops/what-is-devops

Richardson, C. (n.d.). Pattern: Transactional outbox. Microservices.io. https://microservices.io/patterns/data/transactional-outbox.html

Sheppard, V. A. (2019). An introduction to research methods in sociology. Justice Institute of British Columbia. https://pressbooks.bccampus.ca/researchmethods/

Summers, B. L. (2020). Effective methods for software engineering. Auerbach Publishers.

Sumo Logic. (2025, February 17). What is software deployment? https://www.sumologic.com/glossary/software-development

Thales. (n.d.). What is a software maintenance process? 4 types of software maintenance. https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance

Walker, V. (2022, March 16). 14 software architecture design patterns to know. Red Hat. https://www.redhat.com/en/blog/14-software-architecture-patterns

Wells, J. (2025, April 16). Top 12 software quality metrics to measure and why. LinearB. https://linearb.io/blog/software-quality-metrics

Wiggins, A. (n.d.). The twelve-factor app: III. Config. https://12factor.net/config